

 Devin AI Export - PDF Version

Generated on: 11/4/2025, 10:06:08 PM

Source: Devin AI Platform

Overview

Relevant source files

- [README.md](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md> (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md>))
- [package.json](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json> (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json>))

Purpose and Scope

This document provides a high-level overview of the TradeX REST Proxy system, explaining its role as a gateway service in the TradeX trading ecosystem, core capabilities, and architectural design. It covers the system's purpose, key components, and how requests flow through the middleware stack.

For detailed setup instructions, see [Getting Started](#). For configuration details, see [Configuration](#). For security implementation details, see [Security](#).

What is TradeX REST Proxy

TradeX REST Proxy is a high-performance REST API gateway server that serves as a centralized entry point for the TradeX trading system. Built with Node.js, TypeScript, and Express.js, it acts as an intermediary layer between client applications and various backend services, providing authentication, request routing, message transformation, and forwarding capabilities.

The system operates as a middleware gateway that receives HTTP requests from trading clients, authenticates them, applies business logic transformations, and forwards them to appropriate backend services either synchronously via HTTP proxy or asynchronously via Kafka messaging.

Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L1-L28>).

[README.md#L1-L28](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L1-L28) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L1-L28>)

package.json#L1-L5](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L1-L5>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L1-L5>))

Core Capabilities

TradeX REST Proxy provides the following key capabilities:

Capability	Description	Implementation
Multi-Instance Support	Supports both Admin API and Client API instances	Configurable via <code>TRADEX_ENV_INSTANCE</code> environment variable
Authentication & Authorization	JWT-based authentication with RSA key verification	Uses <code>jsonwebtoken</code> dependency and RSA key pairs
Request Routing	Intelligent routing to backend services	HTTP proxy via <code>http-proxy-middleware</code> or Kafka messaging
Scope Management	Dynamic API scope configuration	Managed through <code>src/data/scopeData.json</code>
Device Detection	Client identification and device-specific handling	Express device middleware integration
CORS Support	Configurable Cross-Origin Resource Sharing	Built-in CORS middleware
Message Forwarding	Dual forwarding strategy (HTTP/Kafka)	Configurable per scope and request type
Internationalization	Multi-language support with language detection	Built into request processing pipeline

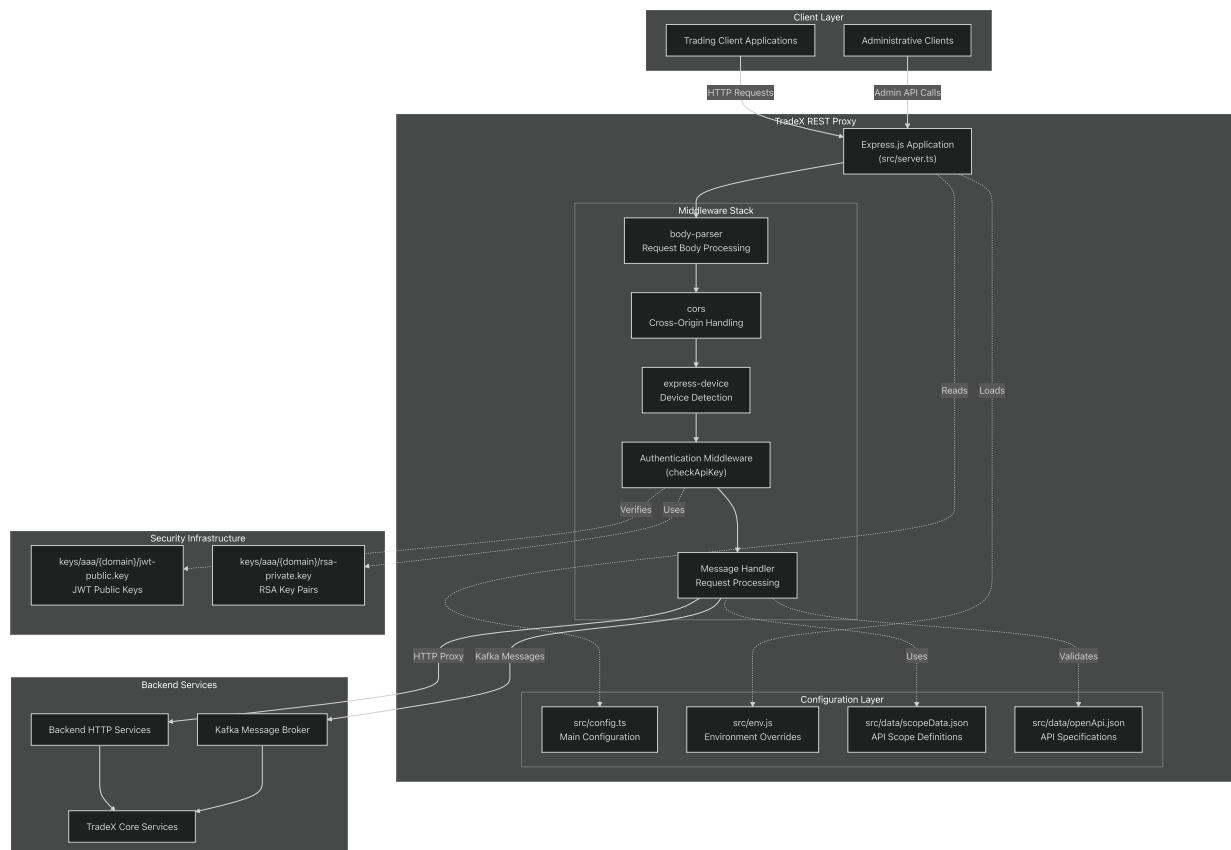
Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L5-L17>)

[README.md5-17](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L5-L17) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L5-L17>)

package.json#L16-L28](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L28>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L28>))

System Architecture

High-Level Component Architecture



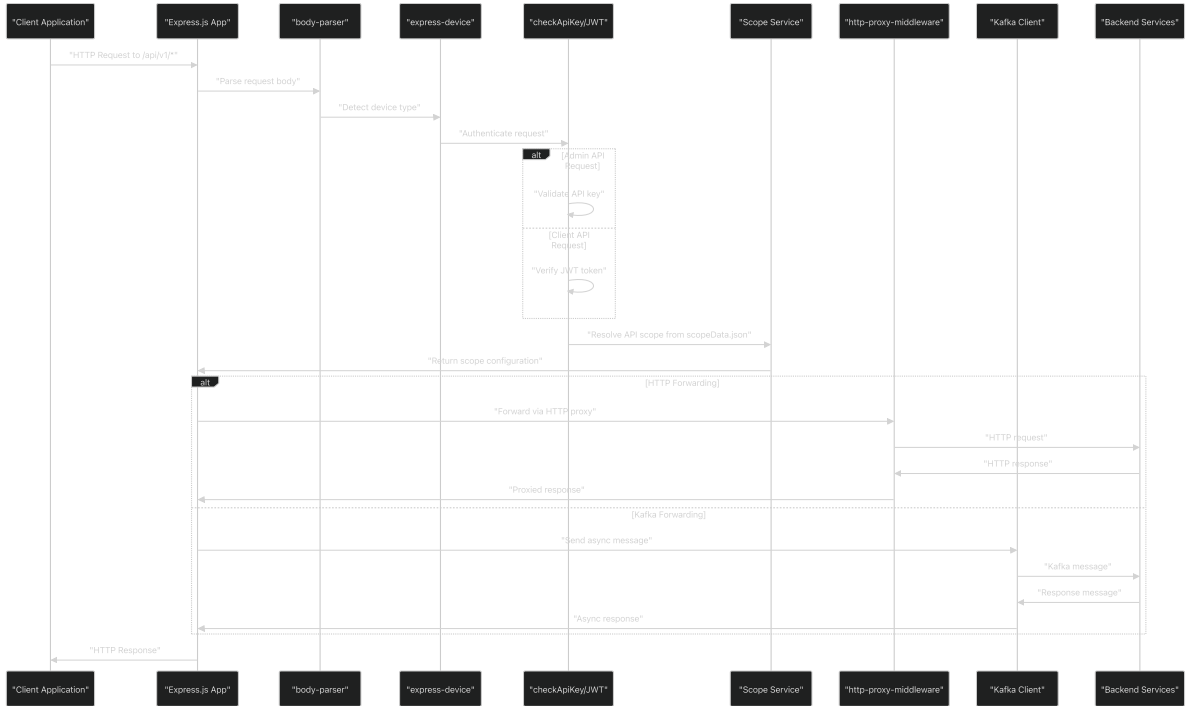
Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L18-L35>)

[README.md#L18-L35](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L18-L35) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L18-L35>)

[README.md#L284-L293](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L284-L293)] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L284-L293>) [([%5B](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L284-L293))

package.json#L16-L28](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L28>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L28>))

Request Processing Flow



Sources: <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L20-L27>
[README.md20-27](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L20-L27) <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L20-L27>
[README.md230-248](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L230-L248) <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L230-L248> <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L230-L248>))

Key Components

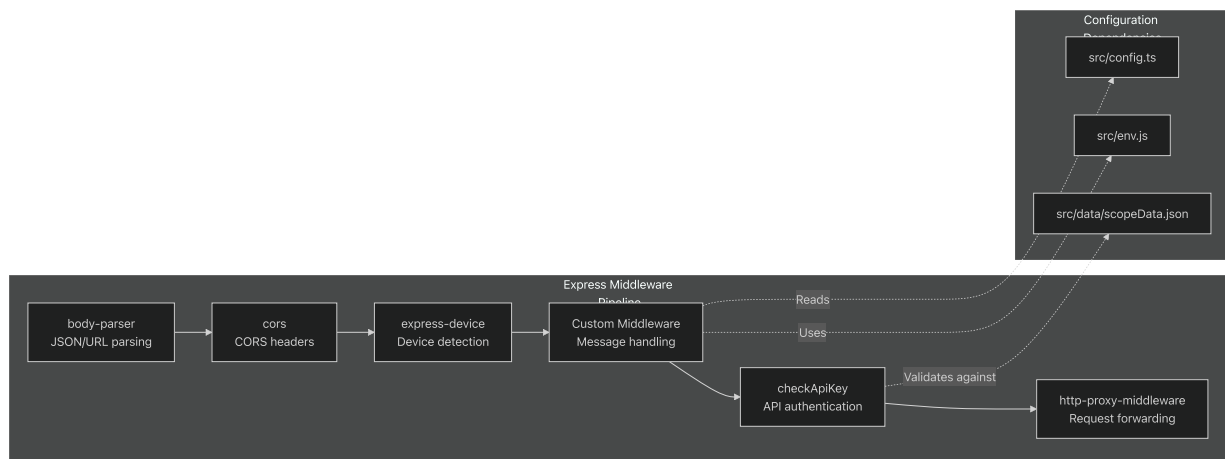
Core Express.js Application

The main application entry point is defined in `src/server.ts` and initialized through `src/index.ts`. The Express application is configured with multiple middleware layers that process incoming requests sequentially.

API Endpoints Structure

Endpoint Pattern	Purpose	Authentication	Example
GET /currentTime	Health check and time service	None	Server timestamp
GET /api/v1/clearCookie/:key	Cookie management	JWT	Clear specific cookies
GET /reInitScope	Scope reinitialization	API Key	Admin operation
POST /api/v1/admin/*	Administrative operations	API Key	Admin functions
POST /api/v1/*	Client API operations	JWT	Trading operations

Middleware Stack Components



Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L29-L35>)

[README.md29-35](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L29-L35) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L29-L35>)

[README.md204-209](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L204-L209)] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L204-L209>) [[%5B](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L204-L209)]

package.json#L16-L24](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L24>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L24>))

Technology Stack

Core Dependencies

The system is built on the following technology foundation:

Component	Technology	Version	Purpose
Runtime	Node.js	16.13.0	JavaScript runtime environment
Language	TypeScript	^5.3.3	Type-safe JavaScript development
Web Framework	Express.js	^4.16.3	HTTP server and middleware
Authentication	jsonwebtoken	^9.0.2	JWT token verification
Proxy	http-proxy-middleware	^2.0.6	HTTP request forwarding
Messaging	tradex-common	git+	Kafka client and utilities
Dependency Injection	typedi	^0.8.0	IoC container

Development and Build Tools

- **TypeScript Compiler:** Compiles TypeScript to JavaScript in `build/src/` directory
- **ESLint + Prettier:** Code linting and formatting with TypeScript support

- **Build System:** NPM scripts for compilation, local development, and production builds
- **Container Support:** Docker multi-stage build with Alpine Linux base image

Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L44>).

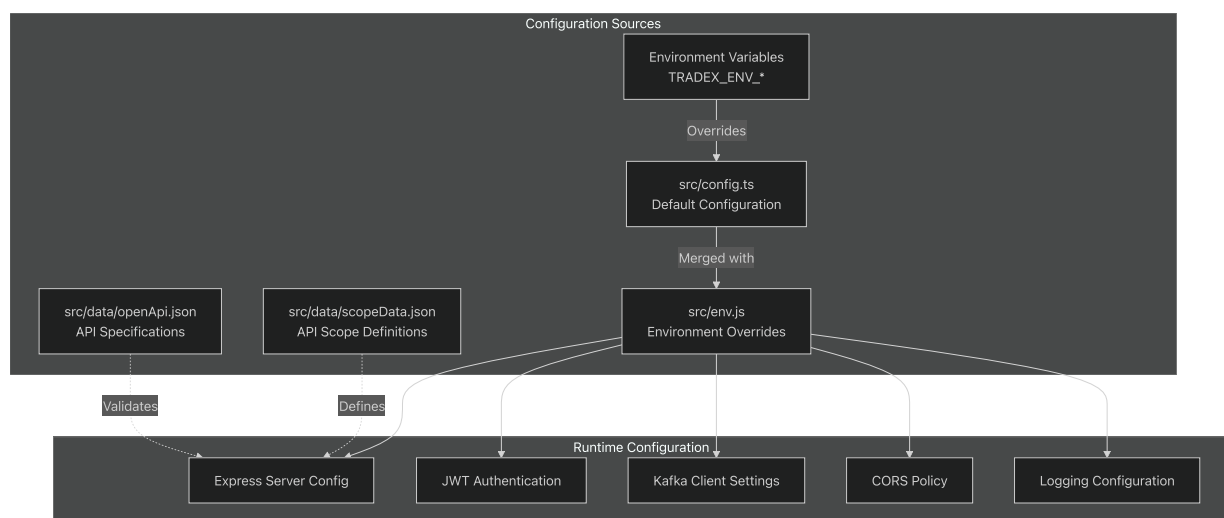
[package.json16-44](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L44) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/package.json#L16-L44>)[

README.md37-44](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L37-L44>)[([%5B](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L37-L44)).

README.md90-99](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L90-L99> (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L90-L99>))

Configuration Architecture

The system uses a layered configuration approach with multiple sources:



Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L157-L183>).

[README.md157-183](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L157-L183) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L157-L183>)[

README.md169-175](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L169-L175> (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/README.md#L169-L175>))

Getting Started

Overview

Relevant source files

- [

setup.sh](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh>
(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh>))

This document provides a comprehensive guide to setting up and deploying the TradeX REST Proxy system. It covers the initial development environment setup, prerequisite verification, and basic configuration steps needed to get the proxy operational.

For detailed configuration options and environment-specific settings, see [Configuration](#).

For information about the underlying architecture and components, see [Architecture](#).

For security setup including key generation and management, see [Security](#).

Prerequisites and Environment

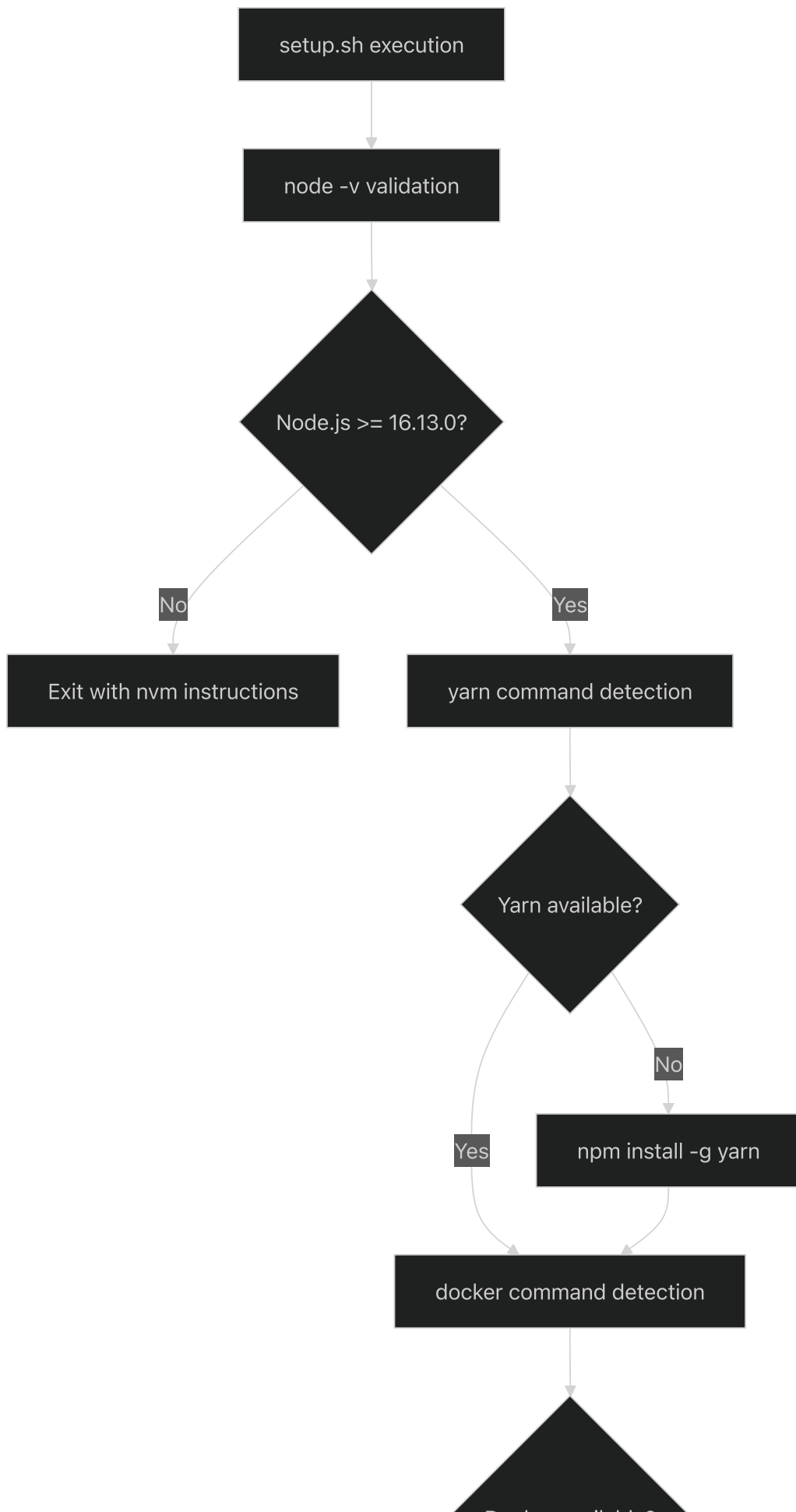
The TradeX REST Proxy requires specific runtime dependencies and development tools to function properly. The system validates these prerequisites during setup to ensure compatibility.

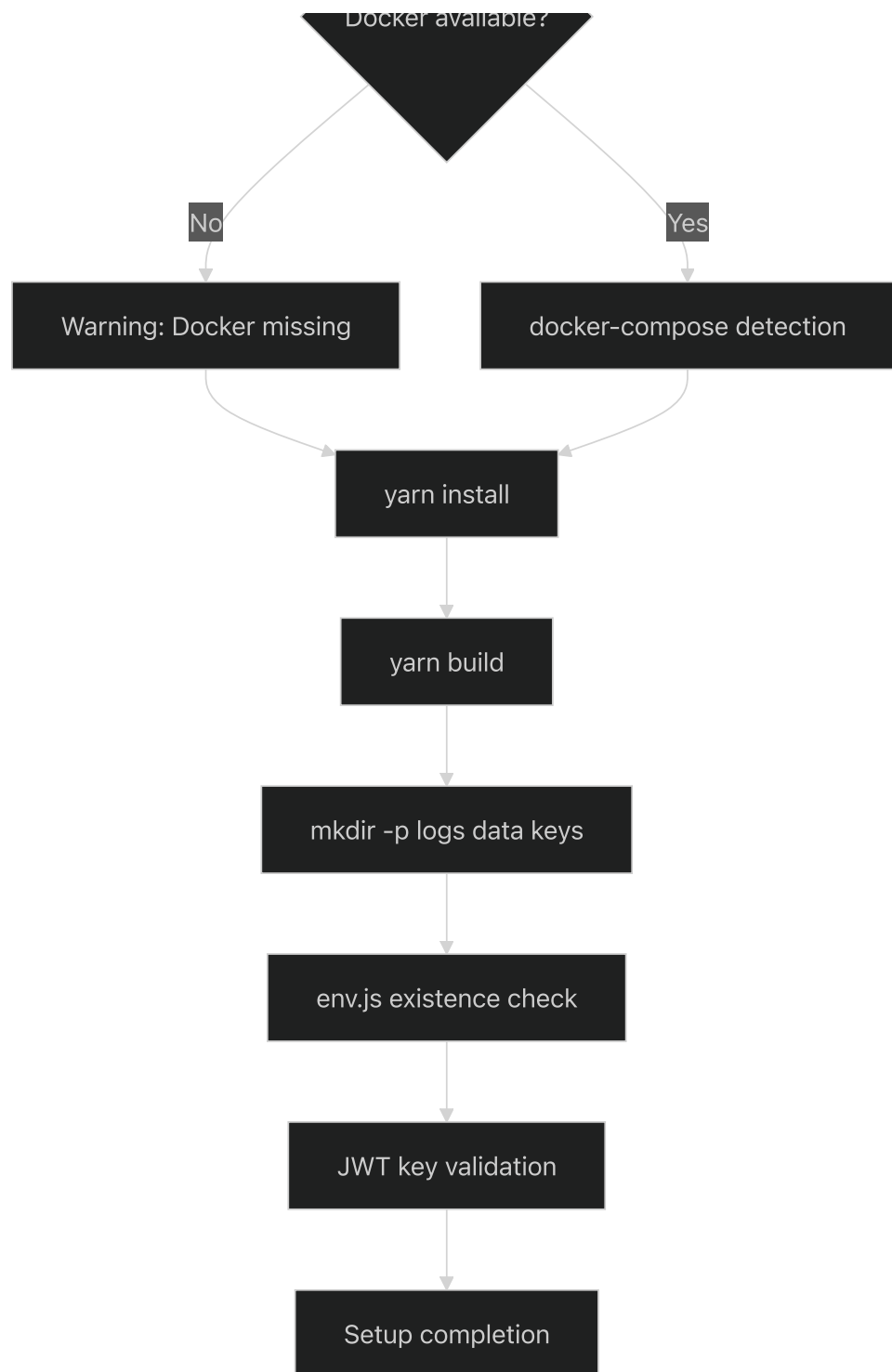
System Requirements

Component	Version	Purpose
Node.js	16.13.0+	Runtime environment
Yarn	Latest	Package management
Docker	Latest	Containerization (optional)
Docker Compose	Latest	Multi-container orchestration (optional)

The setup process automatically detects and validates these dependencies using version checks and command availability tests.

Setup Flow with Prerequisite Validation





Sources: <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L11-L54>.

[setup.sh11-54](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L11-L54) <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L11-L54>

Quick Start Process

The automated setup script `setup.sh` handles the complete initialization workflow, from dependency verification to directory structure creation.

Core Setup Commands

The setup process executes a sequence of commands that prepare the development environment:

```
yarn install
```

```
yarn build
```

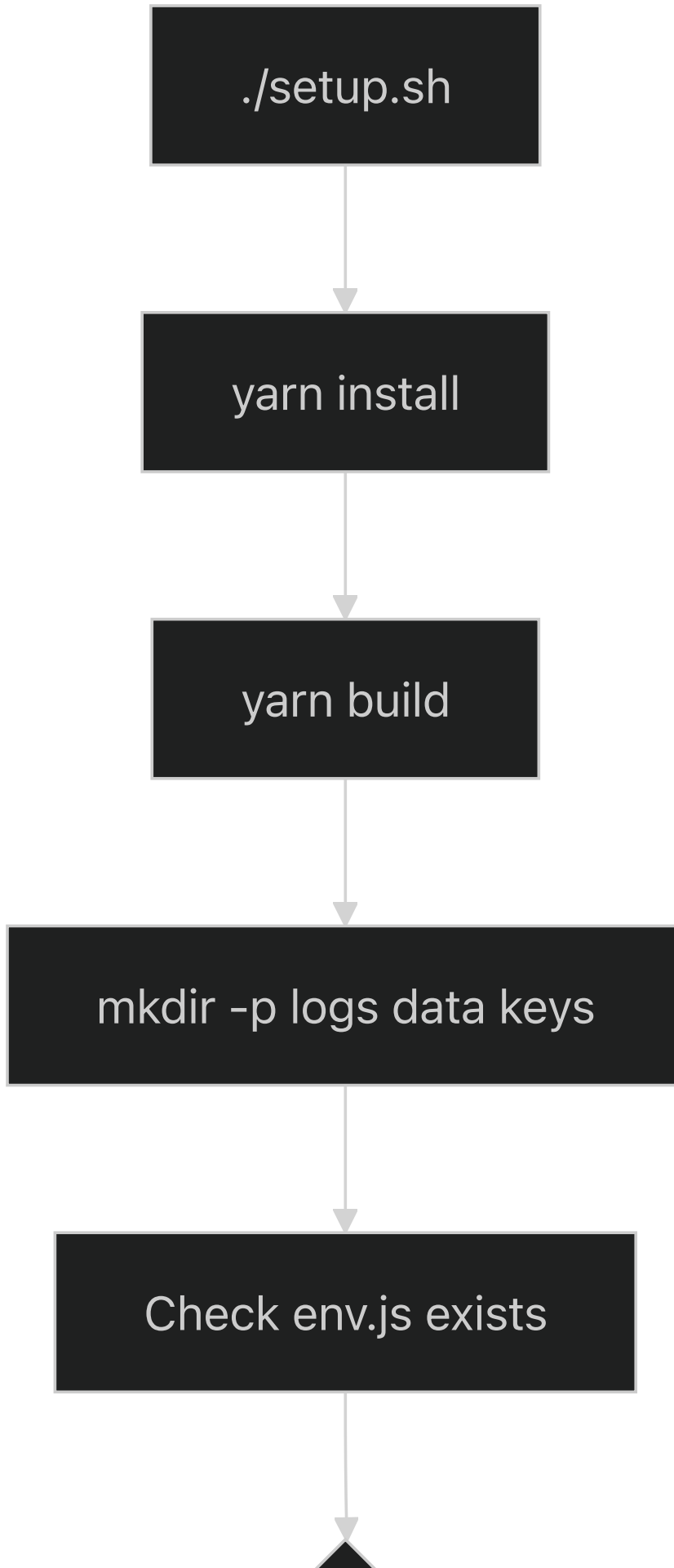
```
mkdir -p logs data keys
```

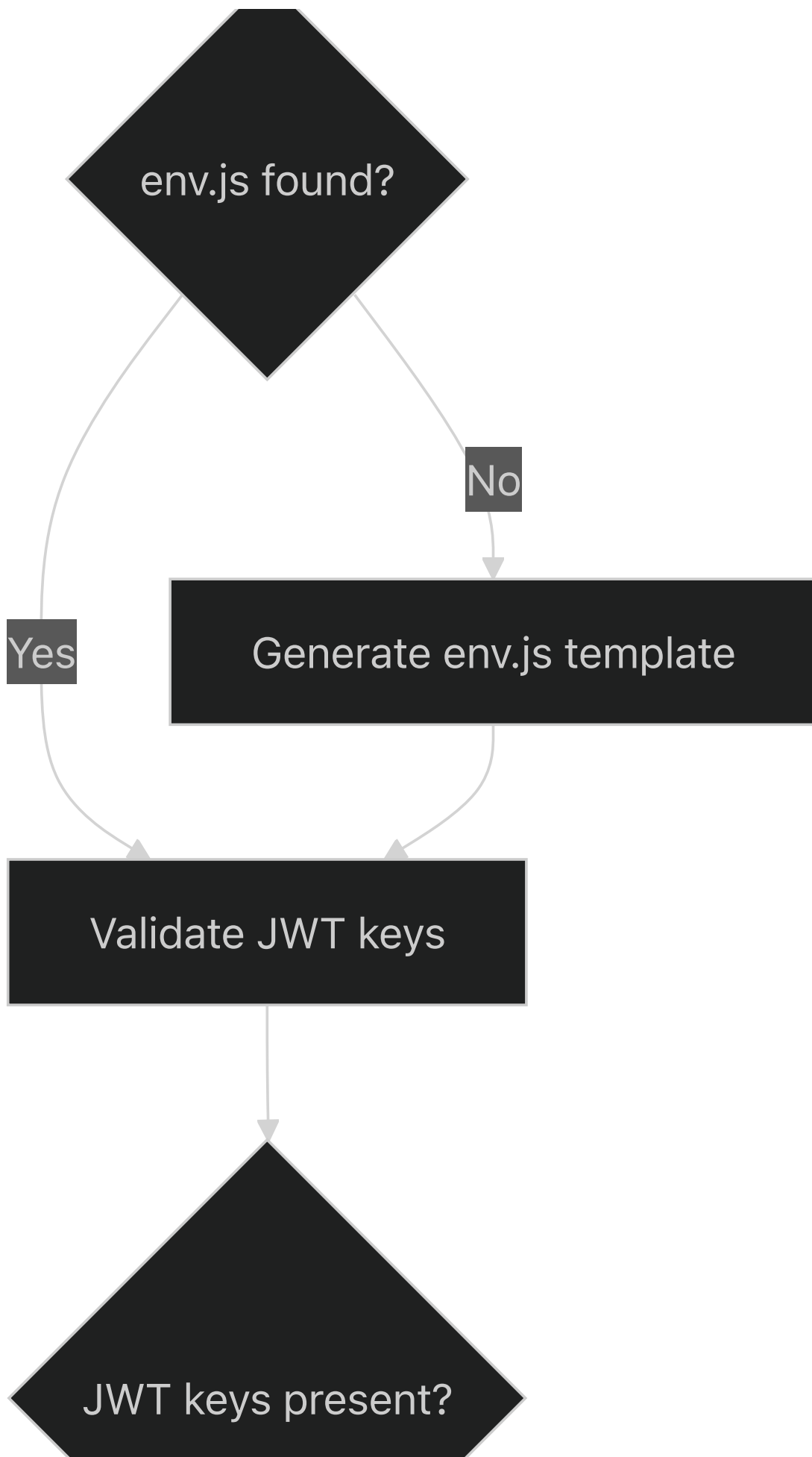
Configuration Template Generation

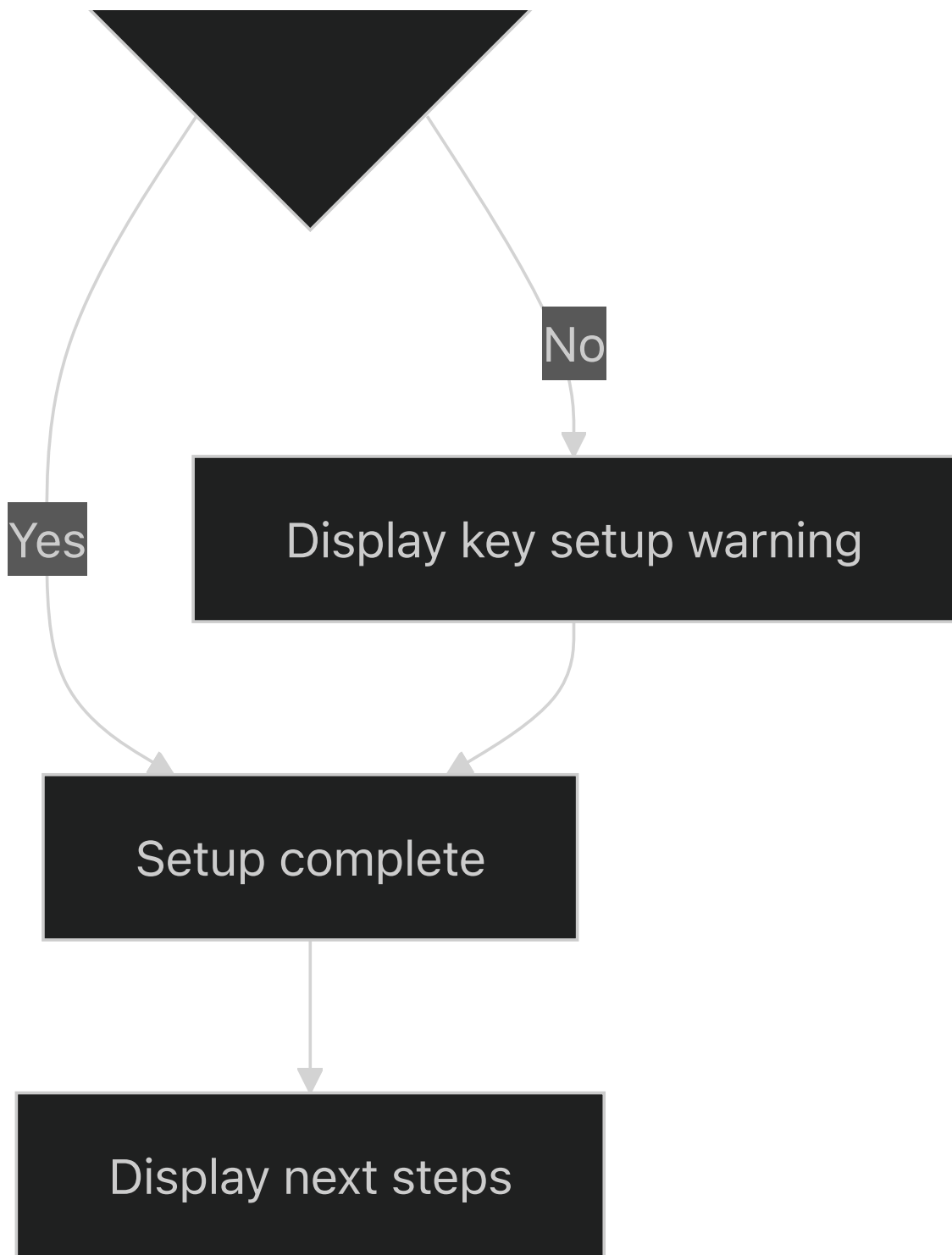
When `env.js` is missing, the setup script generates a functional template with essential configuration parameters:

```
// Generated template structure
conf.kafkaUrls = ["localhost:9092"];
conf.zkUrl = ["localhost:2181"];
conf.topic.configuration = 'configuration-1';
conf.jwt.publicKeyFile = path.join(__dirname, "../keys/jwt-public.key")
```

Setup Script Execution Flow







Sources: <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L57-L126>.

[setup.sh57-126](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L57-L126) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L57-L126>).

Directory Structure and Organization

The setup process creates a standardized directory structure that supports both development and production deployments.

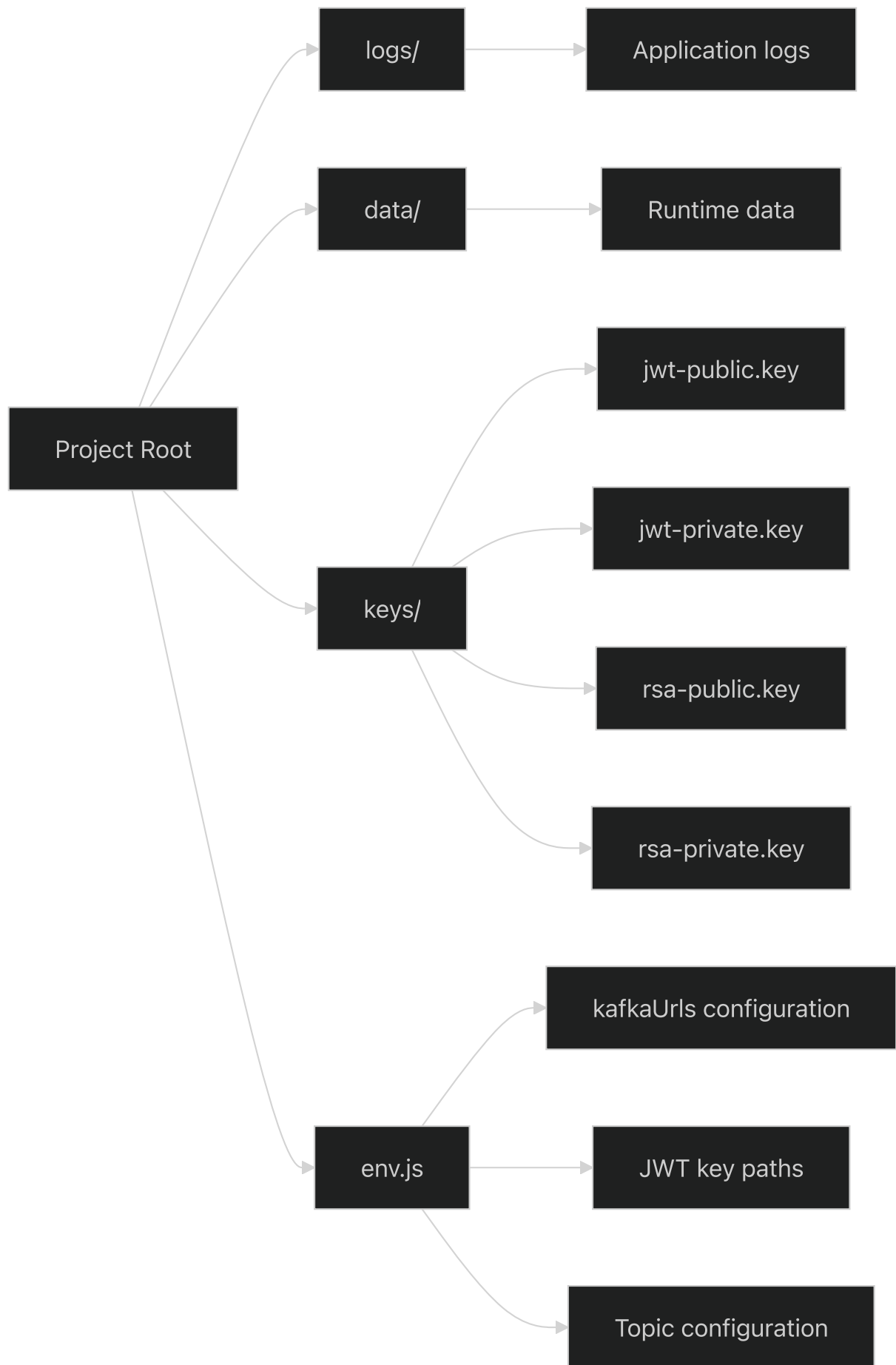
Required Directories

Directory	Purpose	Created By
logs/	Application logging output	setup.sh
data/	Runtime data storage	setup.sh
keys/	Cryptographic key storage	setup.sh

Configuration Files

File	Purpose	Auto-Generated
<code>env.js</code>	Environment configuration	Yes (template)
<code>package.json</code>	Dependency definitions	No
<code>tsconfig.json</code>	TypeScript compilation	No

Directory and Configuration Layout



Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L65-L104>).

[setup.sh65-104](#) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L65-L104>).

Development Workflow Setup

The setup script provides clear next steps for different development scenarios, supporting both local development and containerized deployment.

Local Development Commands

```
yarn watch

yarn run-local

curl http://localhost:3000/currentTime
```

Docker Development Commands

```
make docker-dev

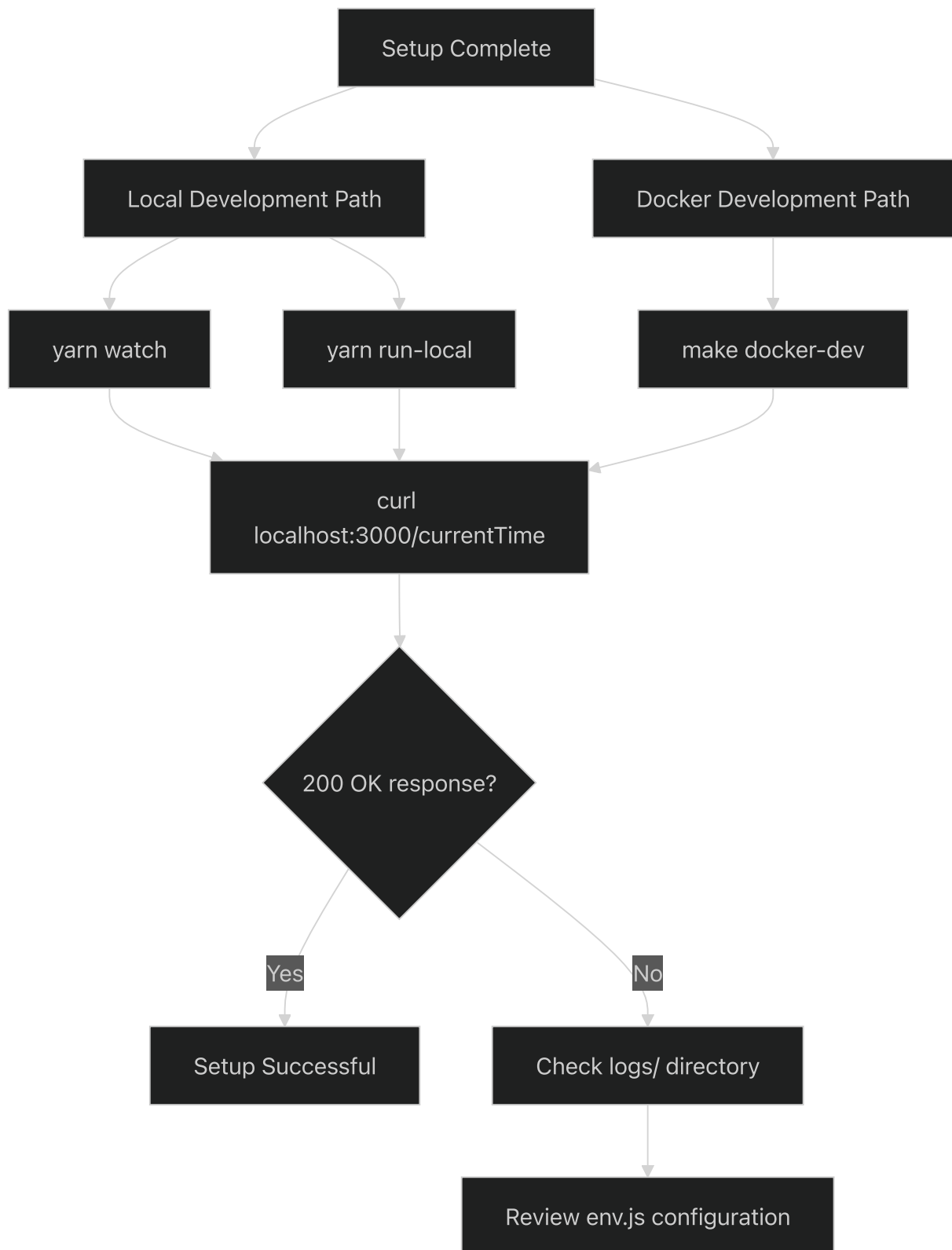
make help
```

Configuration Validation

The setup process performs several validation checks to ensure proper configuration:

1. **JWT Key Validation:** Checks for presence of `keys/jwt-public.key`
2. **Environment Configuration:** Validates `env.js` structure
3. **Service Connectivity:** Provides health check endpoint for testing

Post-Setup Verification Flow



Sources: <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L107-L126>.

[setup.sh107-126](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L107-L126) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L107-L126>)

Troubleshooting Setup Issues

Common setup issues and their resolutions based on the validation checks performed by the setup script:

Node.js Version Issues

- **Problem:** Node.js version below 16.13.0
- **Solution:** Use `nvm install 16.13.0` and `nvm use 16.13.0`
- **Detection:** Version comparison in setup script

Missing Dependencies

- **Problem:** Yarn not installed
- **Solution:** Automatic installation via `npm install -g yarn`
- **Detection:** Command availability check

Docker Warnings

- **Problem:** Docker not available
- **Impact:** Limited to local development only
- **Resolution:** Install Docker Desktop for full functionality

Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L11-L54>).

[setup.sh11-54](#) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L11-L54>).

Initial Setup

Relevant source files

- [`.nvmrc`](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc>)
- [`setup.sh`](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh>)

This document covers the development environment setup process for the TradeX REST Proxy system, including Node.js version requirements, dependency installation, and initial configuration. This setup prepares a local development environment for building and running the proxy service.

For containerized deployment using Docker, see [Docker Deployment](#). For generating cryptographic keys required by the system, see [Key Generation](#). For detailed runtime configuration options, see [Runtime Configuration](#).

Prerequisites

The TradeX REST Proxy requires specific versions of development tools and runtime dependencies to ensure compatibility and proper functionality.

Node.js Version Requirements

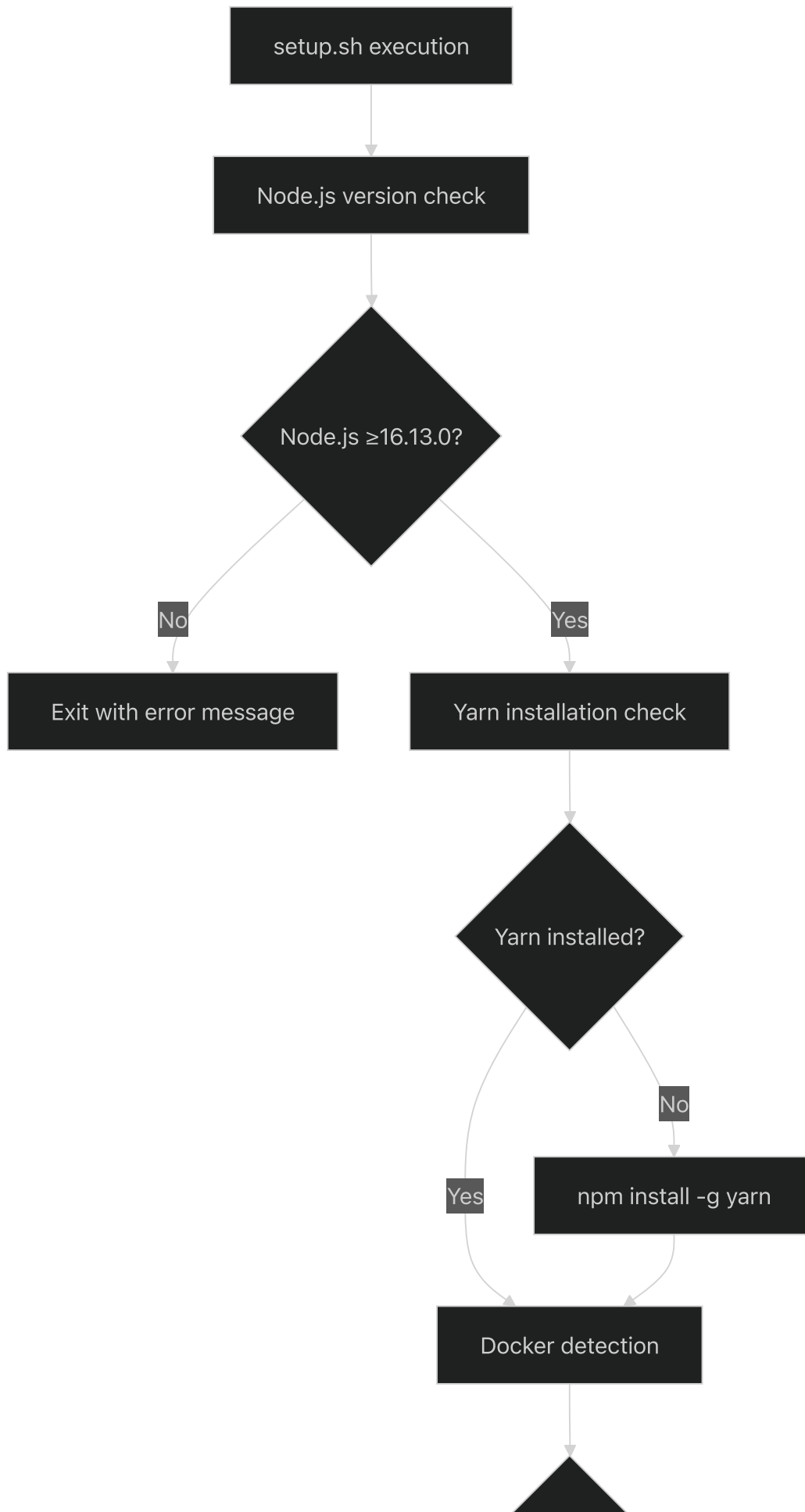
The system requires Node.js version 16.13.0 or later, as specified in

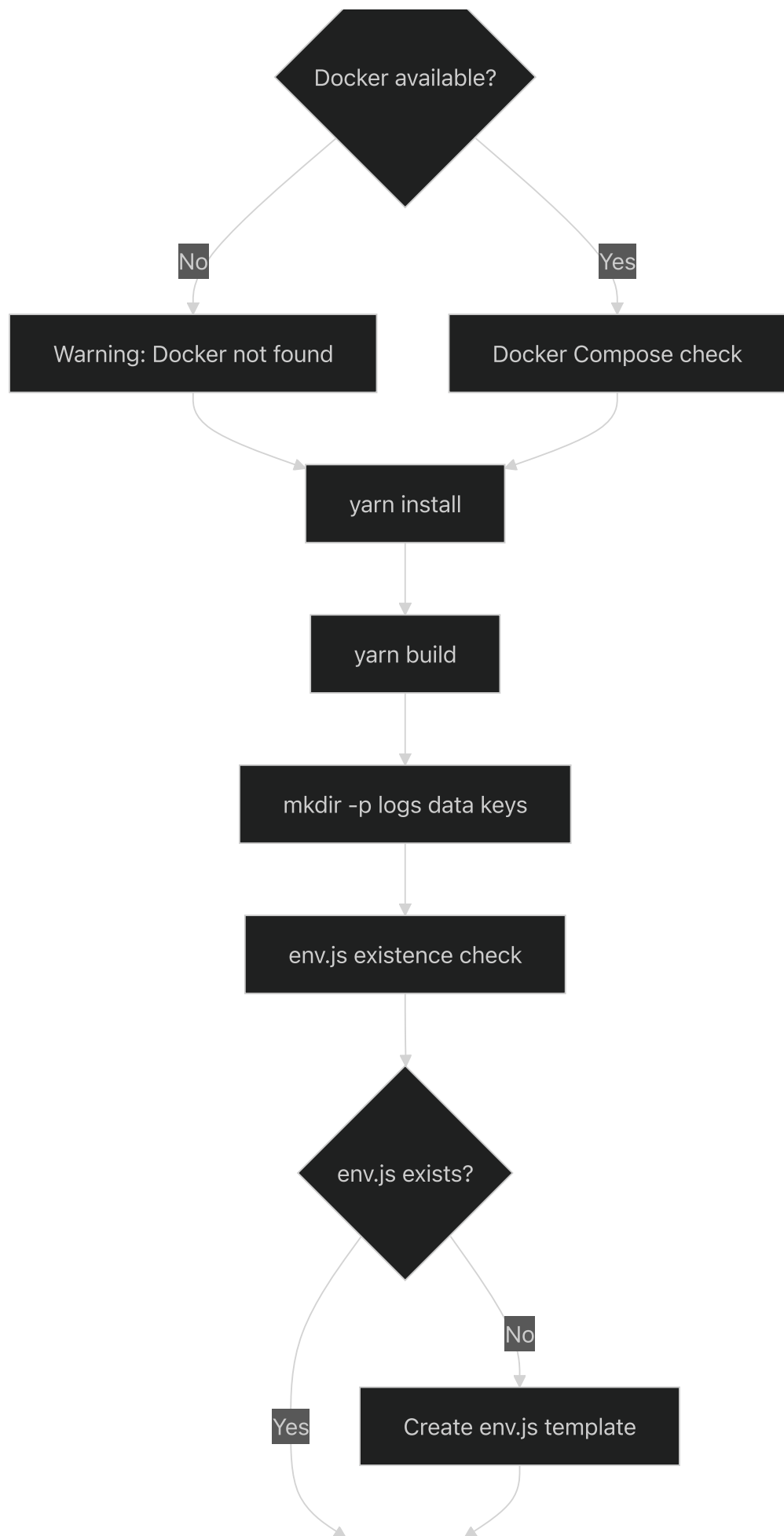
(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc#L1-L1>).

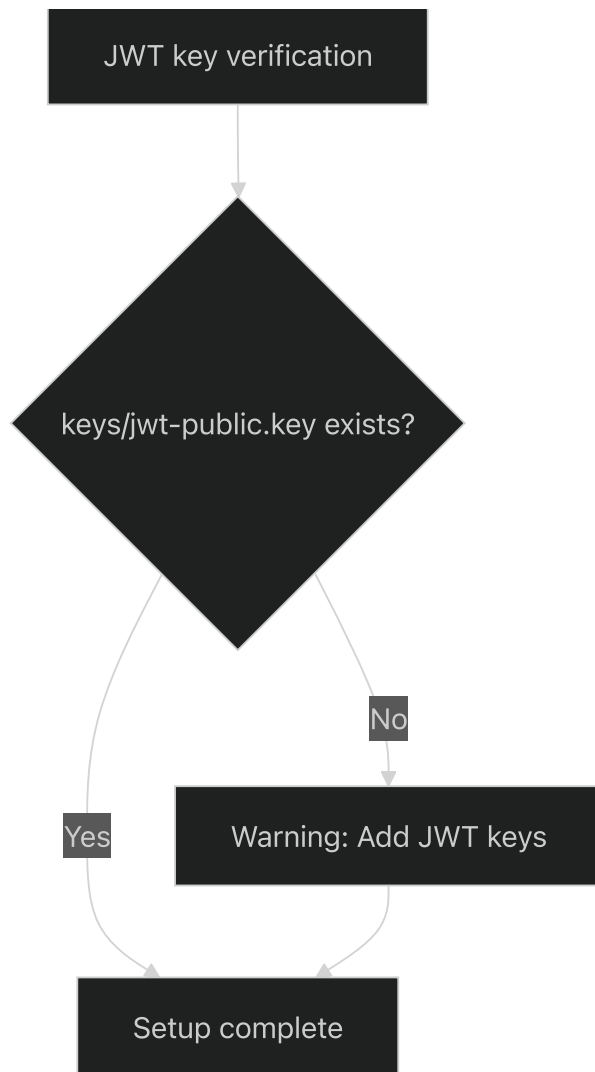
[.nvmrc1](#) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc#L1-L1>). This version requirement ensures compatibility with the TypeScript compilation process and all npm dependencies used by the proxy service.

Tool	Version	Purpose
Node.js	≥16.13.0	JavaScript runtime for development and execution
Yarn	Latest	Package manager for dependency management
Docker	Latest (optional)	Container runtime for dependencies
Docker Compose	Latest (optional)	Multi-container orchestration

Setup Process Flow







Setup Process Flow

Sources: <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L1-L133>

[setup.sh1-133](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L1-L133) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L1-L133>)

Automated Setup Script

The primary setup mechanism is the `setup.sh` script, which automates the entire development environment configuration process.

Script Execution

The setup script performs comprehensive environment validation and configuration:

```
./setup.sh
```

Validation Steps

The script implements several validation checkpoints to ensure environment compatibility:

Node.js Version Validation

(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L20-L28>).

[setup.sh20-28](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L20-L28) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L20-L28>).

implements version comparison logic using `sort -V` to verify the Node.js version meets the minimum requirement specified in[

`.nvmrc1](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc#L1-L1`

(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc#L1-L1>)).

Package Manager Setup

(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L32-L38>).

[setup.sh32-38](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L32-L38) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L32-L38>) detects

Yarn availability and installs it globally via npm if not present, ensuring consistent package management across development environments.

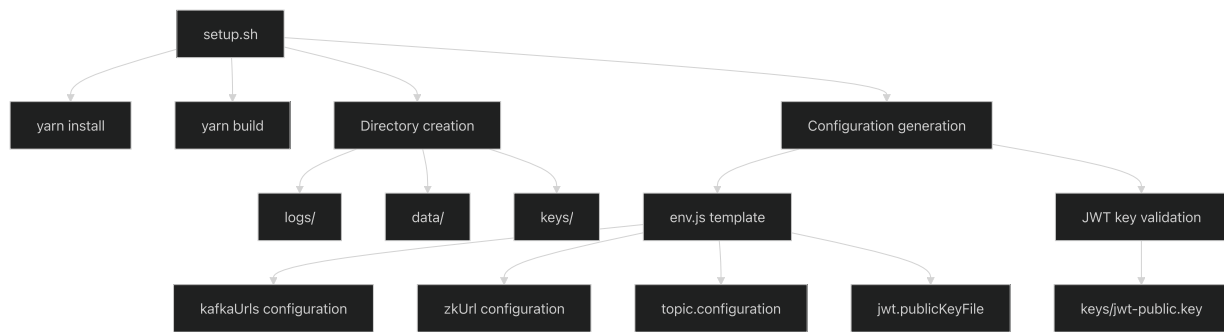
Container Runtime Detection

(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L40-L54>).

[setup.sh40-54](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L40-L54) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L40-L54>) checks

for Docker and Docker Compose availability, providing warnings if not found but allowing development to continue without containerization.

Environment Configuration



Environment Configuration Structure

Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L57-L105>).

[setup.sh57-105](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L57-L105) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L57-L105>).

Configuration Template Generation

When `env.js` is not present, the setup script generates a template configuration file (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L72-L95>).

[setup.sh72-95](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L72-L95) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L72-L95>).

containing:

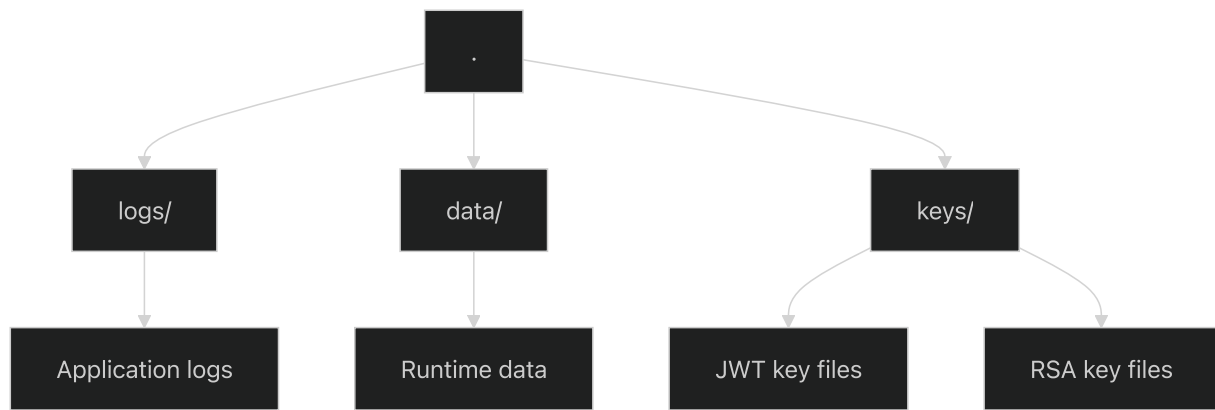
- Kafka broker URLs (`kafkaUrls`)
- ZooKeeper connection strings (`zkUrl`)
- Topic configuration mappings (`topic.configuration`)
- JWT public key file paths (`jwt.publicKeyFile`)
- Domain-specific JWT configurations (`jwt.domains.tradex`)

Manual Setup Steps

For environments where the automated script cannot be used, manual setup requires executing the same validation and configuration steps.

Directory Structure Creation

The following directories must be created in the project root:



Required Directory Structure

Sources: <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L66-L66>.

[setup.sh66](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L66-L66) <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L66-L66>.

Dependency Installation and Build

Manual execution of the build process:

Command	Purpose
<code>yarn install</code>	Install npm dependencies
<code>yarn build</code>	Compile TypeScript sources

Configuration File Setup

If `env.js` does not exist, create it with the template structure shown in

<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L74-L94>.

[setup.sh74-94](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L74-L94) <https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L74-L94>, ensuring proper path references to the `keys/` directory for JWT public key files.

Verification

Health Check Endpoints

After setup completion, verify the installation by testing the health check endpoint:

```
http://localhost:3000/currentTime
```

Development Commands

The setup script provides guidance for starting development processes

(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L113-L119>)

[setup.sh113-119](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L113-L119) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L113-L119>):

Command	Purpose
<code>yarn watch</code>	TypeScript compilation in watch mode
<code>yarn run-local</code>	Local development server
<code>make docker-dev</code>	Docker Compose development environment

Log File Verification

Check the `logs/` directory for application log files to confirm proper logging configuration and service startup.

Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L107-L133>).

[setup.sh107-133](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L107-L133) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/setup.sh#L107-L133>)[

.nvmrc1](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc#L1-L1>
(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.nvmrc#L1-L1>))

Docker Deployment

Relevant source files

- [

dockerfile](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile>
(<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile>))

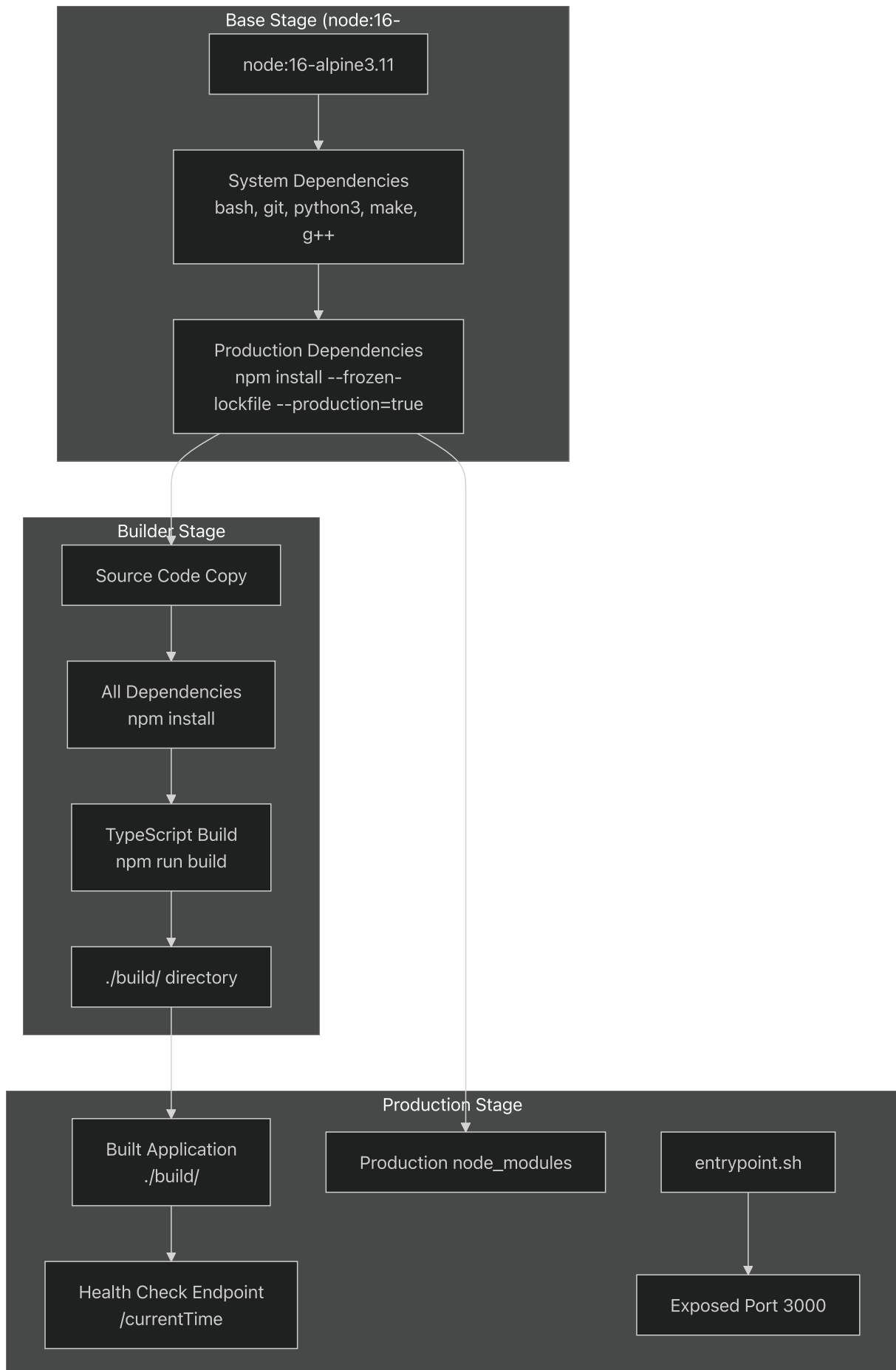
- [
entrypoint.sh](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh> (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh>))

This document covers the containerized deployment of the TradeX REST Proxy system using Docker. It details the multi-stage build process, container configuration, runtime orchestration, and health monitoring setup. For information about initial development setup, see [Initial Setup](#). For cryptographic key management in containers, see [Key Generation](#).

Build Process Overview

The TradeX REST Proxy uses a multi-stage Docker build to optimize the production container size and separate build dependencies from runtime requirements. The build process consists of three distinct stages that progressively refine the container image.

Multi-Stage Build Architecture



Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L1-L57>).

[dockerfile1-57](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L1-L57) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L1-L57>).

The build leverages Docker's multi-stage capabilities to minimize the final image size by excluding development dependencies and build tools from the production container.

Container Configuration

Base Environment Setup

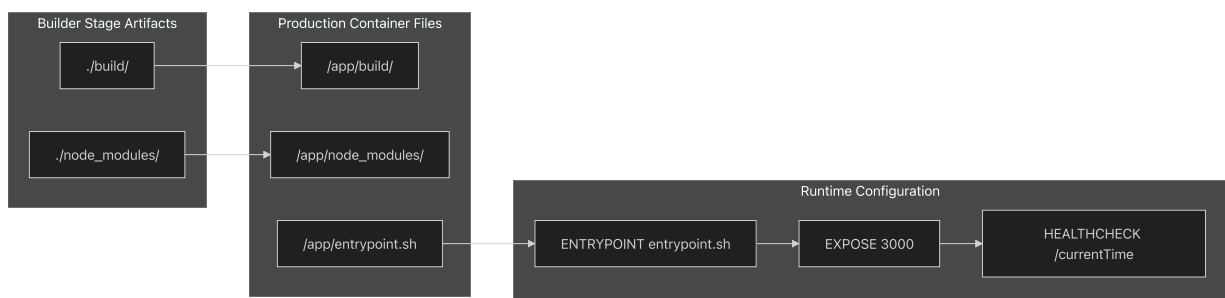
The container environment is configured with specific system dependencies required for Node.js native modules and the TradeX application runtime.

Component	Configuration	Purpose
Base Image	<code>node:16-alpine3.11</code>	Lightweight Node.js 16 runtime
System Packages	<code>bash, git, python3, make, g++</code>	Native module compilation support
Working Directory	<code>/app</code>	Application root directory
Exposed Port	<code>3000</code>	HTTP service endpoint

Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L2-L14>)

[dockerfile2-14](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L2-L14) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L2-L14>).

Production Stage Assembly



Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L34-L56>)

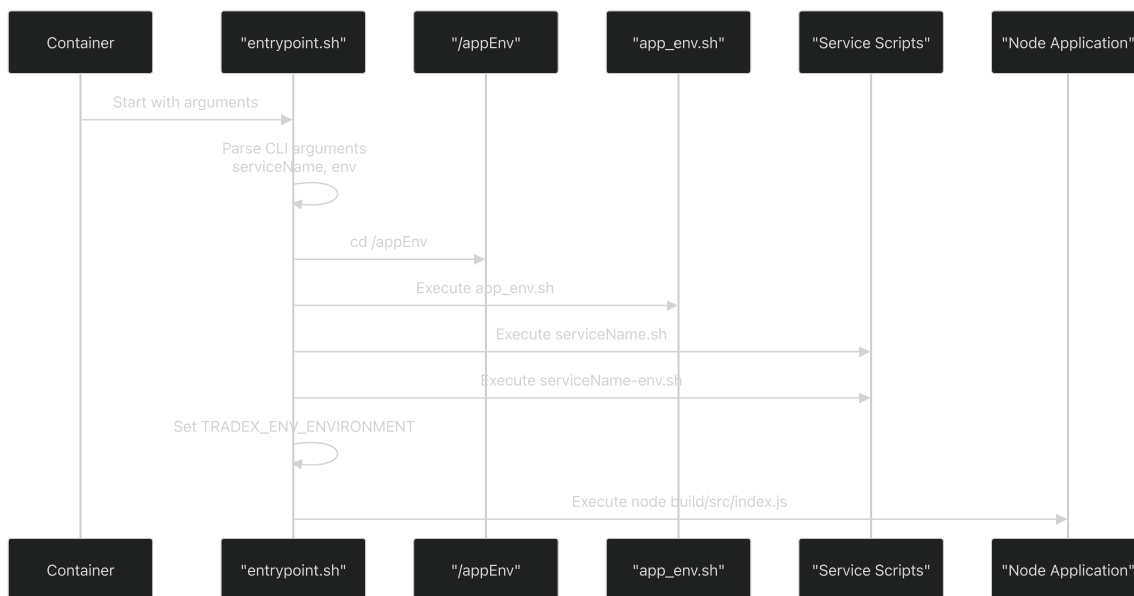
[dockerfile34-56](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L34-L56) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/dockerfile#L34-L56>)

The production stage copies only the compiled application and production dependencies, excluding source code and development tools.

Runtime Orchestration

Entrypoint Script Workflow

The `entrypoint.sh` script provides sophisticated container startup orchestration with environment-specific configuration loading.



Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L1-L90>)

[entrypoint.sh1-90](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L1-L90) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L1-L90>)

Environment Configuration Loading

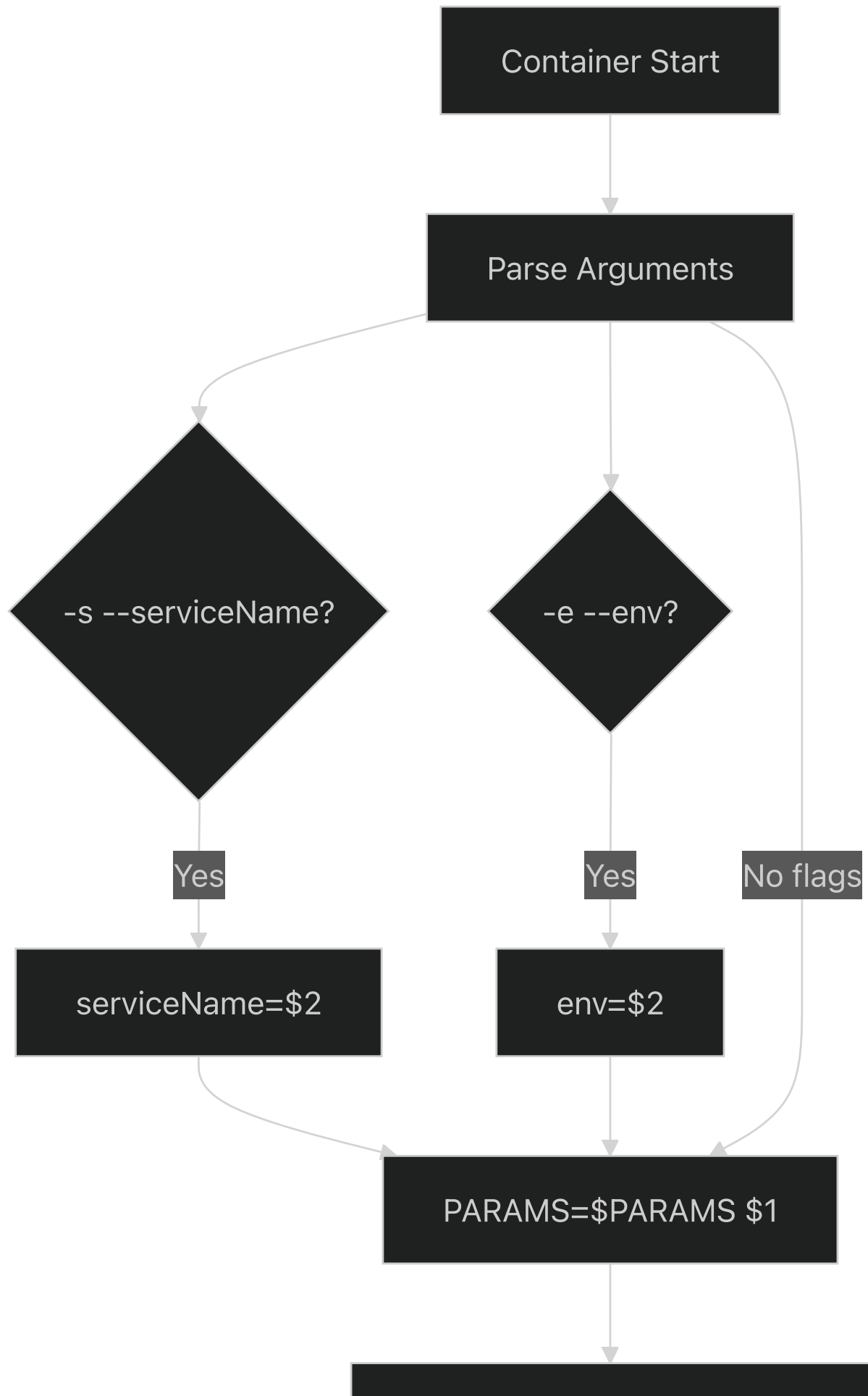
The `entrypoint` script implements a hierarchical configuration loading system:

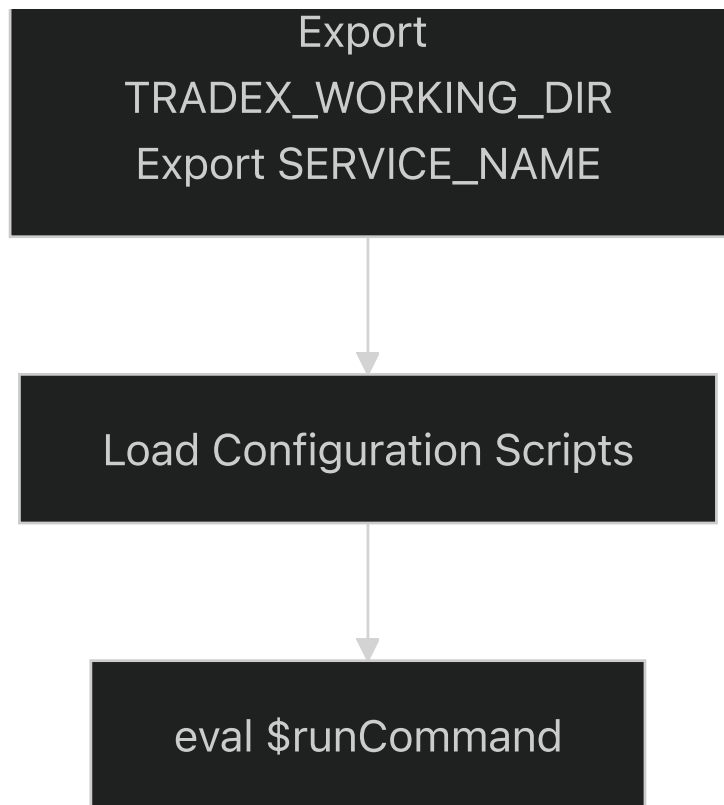
Script Type	File Pattern	Purpose
Global Environment	<code>app_env.sh</code>	Base environment configuration
Service-Specific	<code>{serviceName}.sh</code>	Service-specific settings
Environment-Specific	<code>{serviceName} - {env}.sh</code>	Environment overrides
Local Environment	<code>env.sh</code>	Local configuration overrides

Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L37-L62>).

[entrypoint.sh37-62](#) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L37-L62>)

Argument Processing





Sources: (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L7-L30>)

[entrypoint.sh7-30](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L7-L30) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/entrypoint.sh#L7-L30>)

Health Monitoring

Health Check Implementation

The container includes a built-in health check that validates the application's availability through the `/currentTime` endpoint.

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3
    CMD node -e "require('http').get('http://localhost:3000/currentTi
    | | exit 1 |
|--- | --- |
```

```
**Sources:**[] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74
[dockerfile53-54] (https://github.com/nhsv-tradex/rest-proxy/blob/9502
```

Parameter	Value	Description
interval	30s	Check frequency
timeout	3s	Maximum check duration
start-period	5s	Grace period after container start
retries	3	Consecutive failures before unhealthy

The health check uses Node.js built-in HTTP client to make a lightwei

Deployment Configuration

Environment Variable Setup

The container runtime depends on environment variables set through th

! [Chart 5] (rest-proxy-images/rest-proxy-section3-5.svg)

```
**Sources:**[] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74
[entrypoint.sh34-87] (https://github.com/nhsv-tradex/rest-proxy/blob/9
```

Container Execution Model

The final container execution uses the entrypoint script to orchestra

ENTRYPOINT ["/bin/bash", "/app/entrypoint.sh", "node build/src/index.js"]

This configuration passes `node build/src/index.js` as the final command.

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[dockerfile56] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

The deployment model supports flexible configuration through external configuration files.

Key Generation

Relevant source files

- [create-key.sh] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

This document explains how to generate the cryptographic key pairs required for the system.

For information about how these keys are used in the authentication service, see the [Authentication Service](#) section.

Overview

The TradeX REST Proxy requires two sets of RSA key pairs to function correctly.

- **JWT Key Pair:** Used for signing and verifying JWT tokens in the system.
- **RSA Key Pair:** Used for additional RSA cryptographic operations.

The key generation process creates these pairs using standard SSH key generation tools.

Key Generation Process

Automated Key Generation Script

The system provides an automated script that generates all required key pairs.

Key Generation Workflow

! [Chart 1] (<rest-proxy-images/rest-proxy-section4-1.svg>)


```
Sources: [] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/cr  
[create-key.sh1-10] (https://github.com/nhsv-tradex/rest-proxy/blob/95  
##### Script Execution  
To generate all required cryptographic keys, execute the key generati
```

`./create-key.sh`

This script performs the following operations:

1. **Directory Creation**: Creates the directory structure `keys/aaa`
2. **JWT Key Pair Generation**: Generates the first RSA key pair for
3. **RSA Key Pair Generation**: Generates the second RSA key pair fo

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/cr>

[create-key.sh1-2] (<https://github.com/nhsv-tradex/rest-proxy/blob/950>

Key Generation Details

Directory Structure Creation

![Chart 2] (<rest-proxy-images/rest-proxy-section4-2.svg>)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/cr>

[create-key.sh1] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c>

JWT Key Pair Generation Process

The script generates the JWT key pair using the following sequence:

- Generates initial key pair using `ssh-keygen -N "" -f \$path/rsa`
- Renames private key from `rsa` to `jwt-private.key`
- Renames public key from `rsa.pub` to `jwt-public.key`

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/cr>

[create-key.sh3-5] (<https://github.com/nhsv-tradex/rest-proxy/blob/950>

RSA Key Pair Generation Process

The script generates the RSA key pair using the following sequence:

- Generates second key pair using `ssh-keygen -N "" -f \$path/rsa`
- Renames private key from `rsa` to `rsa-private.key`
- Renames public key from `rsa.pub` to `rsa-public.key`

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/cr>

[create-key.sh8-10] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/create-key.sh>)

Generated Key Files

After successful execution, the following key files will be available

File Name	Type	Purpose
--- --- ---		
`jwt-private.key`	RSA Private Key	JWT token signing
`jwt-public.key`	RSA Public Key	JWT token verification
`rsa-private.key`	RSA Private Key	General RSA operations
`rsa-public.key`	RSA Public Key	General RSA operations

Key Properties

SSH Key Generation Parameters

The script uses `ssh-keygen` with the following parameters:

- `-N ""`: No passphrase (empty passphrase)
- `-f \$path/rsa`: Specifies the output filename

This configuration generates RSA key pairs without password protection

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/create-key.sh>)

[create-key.sh3-8] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/create-key.sh>)

Security Considerations

- ****No Passphrase****: Keys are generated without passphrases for automation
- ****File Permissions****: Ensure proper filesystem permissions are set on generated keys
- ****Deployment Security****: Keys should be securely transferred to deployment environment
- ****Key Rotation****: Consider implementing key rotation procedures for long-term security

Integration with Application

The generated keys integrate with the TradeX REST Proxy's security infrastructure

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/create-key.sh>)

```
[create-key.sh1] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js)
```

Configuration

Overview

Relevant source files

- [[env.js](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js)] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js>)

This document provides a comprehensive guide to configuring the TradeX REST Proxy system. For information about development environment setup and initial configuration, see the [Development Environment Setup](#) document.

Configuration Sources

The TradeX REST Proxy system uses multiple configuration sources that are processed in the following order:



```
! [Chart 1] (rest-proxy-images/rest-proxy-section5-1.svg)
```

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L6>)

[env.js1-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L6>)

Kafka Configuration

The system's Kafka integration is configured through the `env.js` file. The configuration is processed by the `config` function in `env.js`.

Configuration Key	Default Value	Description
<code>kafkaUrls</code>	<code>172.31.145.221:9092</code>	Kafka broker connection string

The Kafka configuration is processed by the `config` function in `env.js`.

[env.js1-4] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L4>)

```
function config(conf) {  
  conf.kafkaUrls = "172.31.145.221:9092";  
  console.log(conf);  
}
```

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en>

[env.js1-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e>

Configuration Processing Flow

The configuration system processes multiple sources and combines them

![Chart 2] (<rest-proxy-images/rest-proxy-section5-2.svg>)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en>

[env.js1-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e>

Runtime Configuration Structure

The configuration system creates a unified configuration object that

Configuration Categories

Category	Purpose	Key Settings
Kafka Integration	Message broker configuration	`kafkaUrls`, to
Authentication	Security and token validation	JWT keys, API key
Forwarding Rules	Request routing logic	HTTP proxy vs Kafka asy
Logging	System monitoring and debugging	Log levels, appenders
Instance Type	Admin vs Client API behavior	Authentication meth

Environment Variable Processing

The system uses a standardized naming convention for environment vari

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en>

[env.js1-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e>

Configuration Integration Points

The configuration system integrates with multiple system components t

![Chart 3] (<rest-proxy-images/rest-proxy-section5-3.svg>)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en>

[env.js1-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e>

Configuration Validation and Error Handling

The configuration system includes validation mechanisms to ensure tha

Validation Process

1. **Schema Validation**: Configuration objects are validated agains
2. **Required Field Checking**: Essential configuration fields are v
3. **Type Coercion**: String values from environment variables are c
4. **Cross-Reference Validation**: Related configuration values are

Error Handling

Configuration errors are handled gracefully with informative error me

[env.js3] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env>

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en>

[env.js1-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e>

Runtime Configuration

Relevant source files

- [
 - combine.sh] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c7>
- [
 - env.js] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e>

Purpose and Scope

This section covers the runtime configuration of the TradeX REST Prox

Environment Configuration

The TradeX REST Proxy system relies on runtime environment configurat

Environment Variables

The system uses environment variables with the `TRADEX\_ENV\_` prefix :

- Service instance type (Admin API vs Client API)
- Logging levels and appenders
- Security settings
- Forwarding rules

Base Configuration Structure

The core configuration is defined in `env.js`, which provides a confi

****Kafka Broker Configuration****

The primary Kafka broker endpoint is configured in[]([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L1-6)

[[env.js#L1-6](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L1-6)]([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L1-6)

! [Chart 1] (<rest-proxy-images/rest-proxy-section6-1.svg>)

****Data Combination Process****

Sources:[]([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/co](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/combine.sh#L1-9)

[[combine.sh#L1-9](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/combine.sh#L1-9)]([https://github.com/nhsv-tradex/rest-proxy/blob/9502c7](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/combine.sh#L1-9)

[env.js#L1-6](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L1-6)]([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env.js#L1-6)

Data Combination Workflow

The runtime configuration involves a multi-step data combination proc

Combination Script Execution

The data combination is orchestrated by `combine.sh`, which performs

1. ****Configuration File Copy****:[]([https://github.com/nhsv-tradex/res](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/combine.sh#L1-9)

- [combine.sh2] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>
- 2. **Directory Navigation**:[] (<https://github.com/nhsv-tradex/rest-p>
- [combine.sh3] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>
- 3. **API Combination Execution**:[] (<https://github.com/nhsv-tradex/r>
- [combine.sh4] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>
- 4. **Return to Root**:[] (<https://github.com/nhsv-tradex/rest-proxy/b>
- [combine.sh5] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>
- 5. **Result Output**:[] (<https://github.com/nhsv-tradex/rest-proxy/bl>
- [combine.sh8] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>

![Chart 2] (<rest-proxy-images/rest-proxy-section6-2.svg>)

Configuration Combination Process

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/co>

[combine.sh1-9] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c7>

Environment Configuration Function

The `env.js` file exports a default configuration function that modif

![Chart 3] (<rest-proxy-images/rest-proxy-section6-3.svg>)

Environment Configuration Function Flow

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en>

[env.js1-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e>

Configuration Properties

Kafka Broker Settings

```

| Property | Value | Source | Purpose |
|---|---|---|---|
| `kafkaUrls` | `"172.31.145.221:9092"` | [ |
| env.js2] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e

```

Configuration Debugging

The configuration function includes console logging[] (https://github.
[env.js3] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/env

Runtime Configuration Loading

The runtime configuration system follows this loading sequence:

1. ****Base Configuration****: The `env.js` configuration function is ca
2. ****Kafka URL Assignment****: The function sets the `kafkaUrls` prope
3. ****Configuration Logging****: The complete configuration is logged t
4. ****Data Combination****: The `combine.sh` script merges various conf
5. ****Unified Output****: A `combined.json` file is generated containin

This process ensures that the application has access to a complete, u

Sources:[] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/en
[env.js1-6] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/e
combine.sh1-9] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74

Development Configuration

Relevant source files

```

- [
    .editorconfig] (https://github.com/nhsv-tradex/rest-proxy/blob/950
- [
    .eslintrc.js] (https://github.com/nhsv-tradex/rest-proxy/blob/9502

```

```
- [
    .eslintrc.json] (https://github.com/nhsv-tradex/rest-proxy/blob/95
- [
    .prettierrc.js] (https://github.com/nhsv-tradex/rest-proxy/blob/95
```

This document covers the development tools configuration for the Trad

For runtime configuration settings, see [Runtime Configuration] (#runt

Development Tools Overview

The TradeX REST Proxy uses a comprehensive development toolchain to m

Development Tool Stack

![Chart 1] (rest-proxy-images/rest-proxy-section7-1.svg)

Sources:[] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.e

[.editorconfig1-9] (https://github.com/nhsv-tradex/rest-proxy/blob/950

.eslintrc.js1-42] (https://github.com/nhsv-tradex/rest-proxy/blob/9502

.eslintrc.json1-3] (https://github.com/nhsv-tradex/rest-proxy/blob/950

.prettierrc.js1-7] (https://github.com/nhsv-tradex/rest-proxy/blob/950

Editor Configuration

The project uses EditorConfig to ensure consistent editor behavior ac

Editor Settings

Setting	Value	Description
indent_style	space	Use spaces instead of tabs
indent_size	2	Two spaces per indentation level
end_of_line	lf	Unix-style line endings (LF)
charset	utf-8	UTF-8 character encoding
insert_final_newline	true	Ensure files end with newline

The EditorConfig applies to all file types (`[\*]`) and ensures consis

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.e>

[.editorconfig3-8] (<https://github.com/nhsv-tradex/rest-proxy/blob/950>

ESLint Configuration

The project uses two ESLint configurations that work together to prov

Primary ESLint Configuration

The main ESLint configuration in `.eslintrc.js` provides the core lin

! [Chart 2] (<rest-proxy-images/rest-proxy-section7-2.svg>)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.e>

[.eslintrc.js2-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>

.eslintrc.js8-11] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>

.eslintrc.js13-14] (<https://github.com/nhsv-tradex/rest-proxy/blob/950>

ESLint Environment Configuration

The ESLint environment is configured to support multiple JavaScript e

- ****Browser****: Enables browser global variables for client-side cod
- ****Node.js****: Enables Node.js global variables and scoping
- ****ES6/ES2021****: Enables modern ECMAScript features and syntax

ESLint Extensions and Parser

The configuration extends several recommended rule sets:

- ``plugin:prettier/recommended``: Integrates Prettier formatting rul
- ``prettier``: Disables ESLint rules that conflict with Prettier
- ``eslint:recommended``: Applies ESLint's recommended rule set

The TypeScript parser (``@typescript-eslint/parser``) enables ESLint to

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.e>

[.eslintrc.js8-14] (<https://github.com/nhsv-tradex/rest-proxy/blob/950>

.eslintrc.js28-31] (<https://github.com/nhsv-tradex/rest-proxy/blob/950>

Google TypeScript Style Integration

The project also includes a secondary ESLint configuration that exten

![Chart 3] (<rest-proxy-images/rest-proxy-section7-3.svg>)

This configuration provides additional TypeScript-specific linting ru

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.e>

[.eslintrc.json2] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502>

Prettier Configuration

Prettier handles automatic code formatting to ensure consistent code

Prettier Settings

Setting	Value	Purpose
semi	true	Require semicolons at statement ends
trailingComma	'all'	Add trailing commas wherever possible
singleQuote	true	Use single quotes instead of double quote
printWidth	150	Maximum line length before wrapping
tabWidth	2	Number of spaces per indentation level

Prettier Integration Flow

![Chart 4] (<rest-proxy-images/rest-proxy-section7-4.svg>)

The 150-character print width provides a balance between readability

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.p>

[.prettierrc.js2-6] (<https://github.com/nhsv-tradex/rest-proxy/blob/95>

Development Workflow Integration

The development tools work together to provide automated code quality

Tool Coordination

![Chart 5](rest-proxy-images/rest-proxy-section7-5.svg)

Configuration Hierarchy

The development tools follow a specific precedence order to avoid con

1. ****EditorConfig****: Provides base editor behavior settings
2. ****Prettier****: Handles code formatting and style
3. ****ESLint****: Validates code quality and TypeScript best practices
4. ****GTS****: Provides additional TypeScript-specific style rules

This hierarchy ensures that formatting is handled by Prettier while E

Sources:[](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/.e>

[.editorconfig1-9](<https://github.com/nhsv-tradex/rest-proxy/blob/950>

.eslintrc.js8-11](<https://github.com/nhsv-tradex/rest-proxy/blob/9502>

.prettierrc.js1-7](<https://github.com/nhsv-tradex/rest-proxy/blob/950>

.eslintrc.json1-3](<https://github.com/nhsv-tradex/rest-proxy/blob/950>

Build Configuration

Relevant source files

- [

 .dockerignore](<https://github.com/nhsv-tradex/rest-proxy/blob/950>

- [

 .eslintignore](<https://github.com/nhsv-tradex/rest-proxy/blob/950>

```
- [
  .gitignore] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74
- [
  package-lock.json] (https://github.com/nhsv-tradex/rest-proxy/blob
```

This page documents the build-time configuration for the TradeX REST

For runtime application configuration, see [Runtime Configuration](#r

Docker Build Configuration

The TradeX REST Proxy uses a comprehensive `.dockerignore` file to co

Build Artifact Exclusions

The Docker build excludes generated build outputs and temporary files

Category	Files/Directories	Purpose
Build Outputs	<code>`build`, `dist`, `*.tsbuildinfo`</code>	Exclude compile
Dependencies	<code>`node_modules`, `jspm_packages/`</code>	Exclude local de
Cache Files	<code>`.npm`, `.eslintcache`, `.cache`, .parcel-cache`</code>	

Sources: [] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74

[.dockerignore7-10] (https://github.com/nhsv-tradex/rest-proxy/blob/95

.dockerignore1-5] (https://github.com/nhsv-tradex/rest-proxy/blob/9502

.dockerignore53-76] (https://github.com/nhsv-tradex/rest-proxy/blob/95

Development Environment Exclusions

Development-specific files and directories are excluded from Docker b

! [Chart 1] (rest-proxy-images/rest-proxy-section8-1.svg)

Sources: [] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74

[.dockerignore13-20] (https://github.com/nhsv-tradex/rest-proxy/blob/9

`.dockerignore22-31]` (<https://github.com/nhsv-tradex/rest-proxy/blob/95>

`.dockerignore97-99]` (<https://github.com/nhsv-tradex/rest-proxy/blob/95>

Security-Sensitive File Exclusions

The build configuration explicitly excludes security-sensitive files

- ****Cryptographic Keys****: ``keys/`, `*.key`, `*.pem`, `*.crt``
- ****Local Configuration****: ``config/local.js`, `config/production.js``
- ****Environment Files****: ``.env*`` files containing sensitive variabl

****Sources:**** [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

`[.dockerignore110-117]` (<https://github.com/nhsv-tradex/rest-proxy/blob>

Code Quality Build Configuration

ESLint Ignore Configuration

The ESLint configuration excludes specific directories from linting d

! [Chart 2] (<rest-proxy-images/rest-proxy-section8-2.svg>)

The ``.eslintignore`` file contains minimal exclusions focusing on dire

- ``node_modules`` - Third-party dependencies
- ``build`` - Compiled output directory

****Sources:**** [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

`[.eslintignore1-3]` (<https://github.com/nhsv-tradex/rest-proxy/blob/950>

Version Control Configuration

Git Ignore Configuration

The ``.gitignore`` file controls which files are excluded from version

Pattern	Purpose	Impact
---	---	---


```
| `/node_modules` | Exclude dependencies | Prevents large dependency
| `/application.log` | Exclude runtime logs | Prevents log pollution
| `/npm-debug.log` | Exclude debug logs | Prevents debug artifacts i
| `/nohup.out` | Exclude background process output | Prevents proces
| `/dist` | Exclude distribution builds | Prevents compiled output i
| `/build` | Exclude build artifacts | Prevents build cache in VCS |
```

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[.gitignore1-7] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

Build Configuration Integration

The three ignore configurations work together to create a clean build

! [Chart 3] (<rest-proxy-images/rest-proxy-section8-3.svg>)

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[.gitignore1-7] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[.eslintignore1-3] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[.dockerignore1-118] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

The build configuration ensures that:

1. **Version Control** contains only source code and essential confi
2. **Code Quality Tools** analyze only relevant source files
3. **Docker Builds** include only necessary files for runtime execut
4. **Security** is maintained by excluding sensitive files from all

This layered approach to build configuration provides optimal perform

Architecture

Overview

Relevant source files

- [

[src/app/middlewares/message-handler.ts](https://github.com/nhsv-t)] (<https://github.com/nhsv-t>)

This document describes the internal architecture of the TradeX REST

For information about security components and cryptographic operation

System Overview

The TradeX REST Proxy is built on Express.js and follows a middleware

! [Chart 1] ([rest-proxy-images/rest-proxy-section9-1.svg](#))

****Core Request Processing Flow****

The system processes requests through a series of middleware function

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>)

[[src/app/middlewares/message-handler.ts31-62](#)] (<https://github.com/nhsv>)

[src/app/middlewares/message-handler.ts64-173](#)] (<https://github.com/nhsv>)

Core Components

Message Handler

The `messageHandler` function serves as the primary Express middlewar

! [Chart 2] ([rest-proxy-images/rest-proxy-section9-2.svg](#))

The message handler creates a unique request context containing:

- `messageId`: Unique identifier combining timestamp, client ID, and
- `req`/`res`: Express request and response objects
- `lang`: Language preference from Accept-Language header
- `rid`: Request ID from headers

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>)

[[src/app/middlewares/message-handler.ts31-62](#)] (<https://github.com/nhsv>)

src/app/middlewares/message-handler.ts19-25] (<https://github.com/nhsv->

Authentication System

The system implements a dual authentication mechanism supporting both

! [Chart 3] (rest-proxy-images/rest-proxy-section9-3.svg)

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>

[src/app/middlewares/message-handler.ts175-256] (<https://github.com/nh>

src/app/middlewares/message-handler.ts92-112] (<https://github.com/nhsv>

src/app/middlewares/message-handler.ts143-161] (<https://github.com/nhs>

Scope Resolution System

The `ScopeService` handles API scope resolution by matching request U

! [Chart 4] (rest-proxy-images/rest-proxy-section9-4.svg)

The scope resolution process includes parameter extraction from URI p

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>

[src/app/middlewares/message-handler.ts138-139] (<https://github.com/nh>

src/app/middlewares/message-handler.ts211-216] (<https://github.com/nhs>

src/app/middlewares/message-handler.ts277-294] (<https://github.com/nhs>

Request Processing Pipeline

Message Preparation and Type Processing

The system transforms incoming HTTP requests into standardized messag

! [Chart 5] (rest-proxy-images/rest-proxy-section9-5.svg)

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>

[src/app/middlewares/message-handler.ts273-356] (<https://github.com/nh>

src/app/middlewares/message-handler.ts401-517] (<https://github.com/nh>

src/app/middlewares/message-handler.ts72-82] (<https://github.com/nhsv>

Kafka Message Forwarding

The `doSendRequest` function handles asynchronous message forwarding

Component	Function	Purpose
---	---	---
`Kafka.getInstance()`	Message broker connection	Singleton Kafka
`sendRequestAsync()`	Async request sending	Send request with t
`getResponse()`	Response parsing	Extract and parse Kafka respo
Transaction ID	`txId` generation	Unique transaction tracking a

![Chart 6] (rest-proxy-images/rest-proxy-section9-6.svg)

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>

[src/app/middlewares/message-handler.ts358-392] (<https://github.com/nh>

Error Handling Architecture

The system implements comprehensive error handling with international

![Chart 7] (rest-proxy-images/rest-proxy-section9-7.svg)

The `returnCode` function provides standardized error responses with

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>

[src/app/middlewares/message-handler.ts394-399] (<https://github.com/nh>

src/app/middlewares/message-handler.ts519-535] (<https://github.com/nh>

Integration Points

External Dependencies

Dependency	Usage	Location
---	---	---
`tradex-common`	Models, Utils, Kafka, Logger, Errors	[src/app/middlewares/message-handler.ts1] (https://github.com/nhsv-t
`express`	Request/Response handling	[src/app/middlewares/message-handler.ts2] (https://github.com/nhsv-trad
`openapi-types`	Schema validation	[src/app/middlewares/message-handler.ts5] (https://github.com/nhsv-trad
`typedi`	Dependency injection container	[src/app/middlewares/message-handler.ts7] (https://github.com/nhsv-trad

Configuration Integration

The message handler integrates with multiple configuration sources:

- `config.ts`: Main application configuration
- Environment variables: Runtime configuration overrides
- `scopeData.json`: API scope definitions
- JWT/RSA key pairs: Cryptographic configuration

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/message-handler.ts4>) (<https://github.com/nhsv-tra>

[src/app/middlewares/message-handler.ts4] (<https://github.com/nhsv-tra>

src/app/middlewares/message-handler.ts11] (<https://github.com/nhsv-tra>

Core Request Processing

Relevant source files

- [src/app/middlewares/message-handler.ts] (<https://github.com/nhsv-t>

This document details the central request processing pipeline in the

For information about the specific middleware components used in this

Request Processing Overview

The core request processing system is implemented in the `messageHand

! [Chart 1] (rest-proxy-images/rest-proxy-section10-1.svg)

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

[src/app/middlewares/message-handler.ts31-62] (<https://github.com/nhsv>

src/app/middlewares/message-handler.ts64-173] (<https://github.com/nhsv>

Request Context Management

The system begins by creating a standardized request context that car

Context Creation

Field	Source	Purpose
---	---	---
`messageId`	Generated sequence	Unique request identifier for 1
`rid`	Request header	Client-provided request ID
`lang`	`accept-language` header	Internationalization support
`req`	Express request object	Original HTTP request
`res`	Express response object	HTTP response handler

The `getMessageId()` function generates unique identifiers using a ti

```
const prefix = `${new Date().getTime()}-${config.clientId};
```

```

**Sources:**[] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74
[src/app/middlewares/message-handler.ts19-25] (https://github.com/nhsv-
src/app/middlewares/message-handler.ts45-52] (https://github.com/nhsv-

```

Authentication Flow

The authentication system supports multiple modes depending on the en

![Chart 2](rest-proxy-images/rest-proxy-section10-2.svg)

```

**Sources:**[] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74
[src/app/middlewares/message-handler.ts92-112] (https://github.com/nhs
src/app/middlewares/message-handler.ts175-256] (https://github.com/nhs

```

JWT Token Processing

The system validates JWT tokens using RSA public key verification wit

- **Scope Group IDs** (`sgIds`): Determines user permissions
- **Refresh Token ID** (`rId`): Links to session management
- **User Data** (`ud`): Contains user profile information

```

payload = await jwtVerify(accessToken, jwtConf.publicKey, {
  algorithms: ['RS256'],
});

```

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[src/app/middlewares/message-handler.ts194-196] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74/src/app/middlewares/message-handler.ts194-196>)

[src/app/middlewares/message-handler.ts202-203] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74/src/app/middlewares/message-handler.ts202-203>)

Scope Resolution System

The scope resolution system determines which backend service should handle the request.

Scope Matching Process

! [Chart 3] (rest-proxy-images/rest-proxy-section10-3.svg)

The `scopeService.findScope()` method returns both the matched scope and the matched pattern template.

- **Parameter Names:** Extracted from URI patterns
- **Parameter Values:** Actual values from the request URI
- **URI Pattern:** The matched pattern template

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[src/app/middlewares/message-handler.ts138-139] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74/src/app/middlewares/message-handler.ts138-139>)

[src/app/middlewares/message-handler.ts211] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74/src/app/middlewares/message-handler.ts211>)

Request Transformation Pipeline

Before forwarding requests to backend services, the system performs a request transformation pipeline.

Parameter Processing

! [Chart 4] (rest-proxy-images/rest-proxy-section10-4.svg)

Type Coercion System

The system implements OpenAPI schema-based type coercion for request parameters.

Schema Type	Transformation Logic
array	JSON parsing or comma-separated splitting


```
| `string` | Direct assignment or JSON stringification |
| `number/integer` | Numeric conversion with validation |
| `boolean` | String-to-boolean conversion |
| `object` | JSON parsing from string |
```

The `processType()` function applies these transformations based on the

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[src/app/middlewares/message-handler.ts274-294] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74/src/app/middlewares/message-handler.ts274-294>)

[src/app/middlewares/message-handler.ts507-517] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74/src/app/middlewares/message-handler.ts507-517>)

[src/app/middlewares/message-handler.ts401-505] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74/src/app/middlewares/message-handler.ts401-505>)

Header Enrichment

The system enriches requests with contextual headers:

```
body.headers['accept-language'] = Utils.getLanguageCode(ctx.lang);
body.headers['user-agent'] = ctx.req.headers['user-agent'];
body.sourceIp = extractedIpAddress;
body.deviceType = ctx.req.device.type;
```

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
[src/app/middlewares/message-handler.ts304-318] (<https://github.com/nh>

Message Forwarding Architecture

The final stage involves forwarding the transformed request to backen

Forwarding Process

![Chart 5] (rest-proxy-images/rest-proxy-section10-5.svg)

The `doSendRequest()` function handles the asynchronous Kafka communi

- **Transaction ID:** Unique identifier for request tracking
- **Topic Routing:** Based on scope configuration
- **Timeout Management:** Configurable request timeouts
- **Response Processing:** Message parsing and HTTP response genera

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
[src/app/middlewares/message-handler.ts358-392] (<https://github.com/nh>
src/app/middlewares/message-handler.ts344-356] (<https://github.com/nhs>

Error Handling and Response Management

The system implements comprehensive error handling with support for i

Error Processing Flow

![Chart 6] (rest-proxy-images/rest-proxy-section10-6.svg)

The `returnCode()` function standardizes error responses with optiona

```
ctx.res.status(status).send({
  code,
  message: config.enableTranslation ? i18n.t(code) : undefined
});
```

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
 [src/app/middlewares/message-handler.ts394-399] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
 src/app/middlewares/message-handler.ts519-535] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

Password Encryption Integration

The system provides optional password encryption for sensitive endpoints.

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
 [src/app/middlewares/message-handler.ts72-82] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
 src/app/middlewares/message-handler.ts113-136] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

Middleware Stack

Relevant source files

- [src/app/middlewares/body-format-verifier.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
- [src/app/middlewares/kafka-forward.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
- [src/app/middlewares/kafka-producer.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
- [src/app/middlewares/request-storage.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
- [src/app/middlewares/uri-parser.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

Purpose and Scope

This document describes the Express.js middleware components that form the core of the system's request processing pipeline.

For information about the overall request processing architecture, see

Middleware Stack Overview

The TradeX REST Proxy uses a layered middleware architecture built on

Middleware Architecture

![[Chart 1](rest-proxy-images/rest-proxy-section11-1.svg)]

Sources:[](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src>)

[src/app/middlewares/request-storage.ts4-13] ([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/request-storage.ts#L4-L13))

[src/app/middlewares/body-format-verifier.ts3-11] ([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/body-format-verifier.ts#L3-L11))

[src/app/middlewares/uri-parser.ts4-6] ([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/uri-parser.ts#L4-L6))

[src/app/middlewares/kafka-forward.ts27-51] ([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-forward.ts#L27-L51))

[src/app/middlewares/kafka-producer.ts7-95] ([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-producer.ts#L7-L95))

Request Storage Middleware

The `storeRequest` middleware is the entry point of the processing pipeline.

Request Context Storage

Property	Type	Purpose
---	---	---
`session_id`	UUID	Unique identifier for request tracking
`requestStack`	Object	Application-local storage for request context

The middleware assigns a UUID to each request and stores the request, context, and response in the request stack.

![[Chart 2](rest-proxy-images/rest-proxy-section11-2.svg)]

Sources:[](<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src>)

[src/app/middlewares/request-storage.ts1-15] ([https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src](https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/request-storage.ts#L1-L15))

Body Format Verifier

The `verifyFormat` middleware standardizes request body formats by co

Format Transformation

The middleware performs the following transformation:

- ****Input****: `[item1, item2, item3]`
- ****Output****: `{ items: [item1, item2, item3] }`

![Chart 3](rest-proxy-images/rest-proxy-section11-3.svg)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>

[src/app/middlewares/body-format-verifier.ts3-11] (<https://github.com/>

URI Parser Middleware

The `parseURI` middleware constructs a standardized URI format for ro

URI Construction

The parser creates a `parsedURI` property with the format: `{method}:`

![Chart 4](rest-proxy-images/rest-proxy-section11-4.svg)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/sr>

[src/app/middlewares/uri-parser.ts1-8] (<https://github.com/nhsv-tradex>

Kafka Forward Middleware

The `forward` middleware handles the complex process of preparing and

Forward Configuration Interface

The middleware accepts an `IForwardConfig` object that defines forwar

Property	Type	Values	Purpose
topic	string	Kafka topic name	Target topic for message rou

```
| `uri` | string | Service URI | Backend service identifier |
| `secToken` | string | "decode", "verify", null | JWT token process
```

Request Processing Flow

![Chart 5](rest-proxy-images/rest-proxy-section11-5.svg)

IP Address Extraction

The middleware extracts client IP addresses from multiple header sources:

1. `tx-source-ip` header
2. `x-forwarded-for` header
3. `connection.remoteAddress`

IPv6 addresses are detected using regex validation and processed accordingly.

JWT Token Processing

When `secToken` is configured, the middleware:

- Strips the "jwt " prefix from the token
- Validates the token using RSA256 algorithm
- Converts the payload to internal format using `Utils.convertToken`
- Handles token expiration based on the `secToken` mode

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-forward.ts#L117>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-forward.ts#L117>)

Kafka Producer Middleware

The `sendToKafka` middleware provides an alternative Kafka forwarding mechanism.

Field Type Coercion

The middleware performs automatic type conversion based on configuration:

Type	Conversion	Example
integer	Math.floor()	"3.7" → 3
boolean	/true/i.test()	"True" → true

```

    | `array` | String wrapping | "value" → \["value"\] |

##### Processing Pipeline

![[Chart 6](rest-proxy-images/rest-proxy-section11-6.svg)]

##### Error Response Handling

The middleware translates error codes using the i18n system and maps

Sources:[] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src

[src/app/middlewares/kafka-producer.ts1-98] (https://github.com/nhsv-t

#### Integration and Configuration

The middleware stack integrates with several configuration systems an

##### Dependencies

    | Component | Source | Purpose |
    |---|---|---|
    | `tradex-common` | External library | Logging, Kafka, utilities |
    | `config` | Internal module | Runtime configuration |
    | `jwtVerify` | Promisified module | JWT token validation |
    | `uuid` | External library | Unique identifier generation |

##### Message ID Generation

Both Kafka middlewares use time-based message ID generation with clie

```

```
{timestamp}-{clientId}-{incrementalId}-{requestId}
```

This ensures globally unique message tracking across the distributed

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-forward.ts#L8-19>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-producer.ts#L4>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-producer.ts#L4>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-producer.ts#L4>)

[src/app/middlewares/kafka-forward.ts#L8-19] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-forward.ts#L8-19>)

[src/app/middlewares/kafka-producer.ts#L4] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/kafka-producer.ts#L4>)

Authentication Middleware

Relevant source files

- [

src/app/middlewares/check-apikey.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts>)

This document covers the API key authentication middleware component

For information about JWT-based authentication used for client APIs,

Purpose and Implementation

The authentication middleware provides API key validation for adminis

[src/app/middlewares/check-apikey.ts#L4-11] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L4-11>)

The middleware follows Express.js middleware conventions, accepting `

Middleware Function Structure

The `checkApiKey` middleware implements a straightforward authenticat

! [Chart 1] (<rest-proxy-images/rest-proxy-section12-1.svg>)

****Middleware Decision Flow**** Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L4-11>)

[src/app/middlewares/check-apikey.ts#L4-11] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L4-11>)

The middleware performs header validation at[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L4-11>)

[src/app/middlewares/check-apikey.ts5] (<https://github.com/nhsv-tradex>

src/app/middlewares/check-apikey.ts6] (<https://github.com/nhsv-tradex/>

Integration with Express Middleware Stack

The `checkApiKey` middleware integrates into the broader Express.js m

![Chart 2](rest-proxy-images/rest-proxy-section12-2.svg)

****Authentication Middleware in Request Processing Chain**** Sources:[] (

[src/app/middlewares/check-apikey.ts1-13] (<https://github.com/nhsv-tra>

Configuration Dependencies

The middleware relies on the system configuration for the API key val

Component	Source	Purpose
`config.apiKey`	[	
src/app/middlewares/check-apikey.ts2]	(https://github.com/nhsv-trad	
Import statement	[	

src/app/middlewares/check-apikey.ts2] (<https://github.com/nhsv-tradex/>

The `config` object imported at[] (<https://github.com/nhsv-tradex/rest>

[src/app/middlewares/check-apikey.ts2] (<https://github.com/nhsv-tradex>

Authentication Flow

The authentication process follows this sequence:

![Chart 3](rest-proxy-images/rest-proxy-section12-3.svg)

****API Key Authentication Sequence**** Sources:[] (<https://github.com/nhs>

[src/app/middlewares/check-apikey.ts4-11] (<https://github.com/nhsv-tra>

Error Response Format

When authentication fails, the middleware returns a standardized JSON

```
{  
  "message": "Unauthorized",  
  "code": 403  
}
```

This error format at[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L6>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L6>)

Security Characteristics

The authentication middleware implements several security features:

- ****Header-based Authentication****: Uses the `apikey` header for cre
- ****Fail-secure Design****: Defaults to rejection if the API key does
- ****Immediate Response****: Returns 403 status immediately on authent
- ****No Information Leakage****: Provides generic "Unauthorized" messa

The middleware's simple design at[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L4-L11>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L4-L11>)

[src/app/middlewares/check-apikey.ts#L4-L11] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L4-L11>)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L1-L13>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L1-L13>)

[src/app/middlewares/check-apikey.ts#L1-L13] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts#L1-L13>)

Data Models

Relevant source files

- [src/app/models/admin/AWS.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/models/admin/AWS.ts>)
- [src/app/models/admin/Authentication.ts] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/models/admin/Authentication.ts>)

This document describes the API data structures and response schemas

For information about the authentication mechanisms that use these mo

Purpose and Scope

The data models in the TradeX REST Proxy serve multiple purposes:

- ****API Documentation****: Provide OpenAPI/Swagger schema definitions
- ****Request Validation****: Define expected request formats for input
- ****Response Standardization****: Ensure consistent response formats
- ****Type Safety****: Support TypeScript type checking and development

The models are organized by functional area and are primarily located

Model Architecture Overview

The data models in the system follow the OpenAPI 3.0 specification fo

![[Chart 1](rest-proxy-images/rest-proxy-section13-1.svg)]

****Sources:**** `src/app/models/admin/Authentication.ts`, `src/app/model

Authentication Data Models

The authentication models define the structure of login responses and

AdminLoginResponse Schema

The `AdminLoginResponse` model defines the structure returned when an

![[Chart 2](rest-proxy-images/rest-proxy-section13-2.svg)]

Property	Type	Description
---	---	---
`accessToken`	string	Token to access all other APIs in system
`refreshToken`	string	Token to renew access_token when it's e
`userInfo`	AdminInfoResponse	Admin user information object

RefreshTokenResponse Schema

The `RefreshTokenResponse` model defines the structure returned when

Property	Type	Description
---	---	---
`accessToken`	string	New access token to call API

****Sources:**** [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

[\\`src/app/models/admin/Authentication.ts1-24] (<https://github.com/nhs>)

AWS Integration Data Models

The AWS integration models define the structure for S3 file upload op

SignedDataResponse Schema

The `SignedDataResponse` model defines the structure returned when ge

![[Chart 3](rest-proxy-images/rest-proxy-section13-3.svg)]

Property	Type	Description
---	---	---
`key`	string	The file name including the path that the file wi
`bucket`	string	The bucket that file will be uploaded to
`Policy`	string	The post policy of the request
`X-Amz-Signature`	string	Data returned from S3 Service
`X-Amz-Date`	string	Data returned from S3 Service
`X-Amz-Credential`	string	Data returned from S3 Service
`X-Amz-Algorithm`	string	Data returned from S3 Service

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[\\`src/app/models/admin/AWS.ts1-29] (<https://github.com/nhsv-tradex/re>)

Model Integration with Request Processing

The data models integrate with the broader request processing system

![[Chart 4](rest-proxy-images/rest-proxy-section13-4.svg)]

Schema Validation Flow

1. **Request Parsing**: Incoming requests are parsed and matched aga
2. **Type Coercion**: Data types are converted according to OpenAPI
3. **Validation**: Request structure and content are validated again
4. **Response Formatting**: Outgoing responses are formatted accordi

Model Reference Resolution

The models use OpenAPI `\$ref` syntax to reference other schema defini

Sources: (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74
 [\`src/app/models/admin/Authentication.ts16] (https://github.com/nhsv-
 \`src/app/models/admin/AWS.ts1-29] (https://github.com/nhsv-tradex/res
 #### OpenAPI Integration

All data models are designed to integrate seamlessly with the OpenAPI

- **Schema Location:** All schemas are defined under `components.sche
- **Reference Format:** Cross-references use the `#/components/sche
- **Documentation:** Each property includes human-readable descript
- **Type Specification:** All properties specify their data types a

This integration enables automatic API documentation generation, clie

Sources: (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74
 [\`src/app/models/admin/Authentication.ts1-24] (https://github.com/nhs
 \`src/app/models/admin/AWS.ts1-29] (https://github.com/nhsv-tradex/res

Security

Overview

Relevant source files

- [

 keys/aaa/tradex/jwt-private.key] (https://github.com/nhsv-tradex/r
- [

 keys/aaa/tradex/jwt-public.key] (https://github.com/nhsv-tradex/re

This document provides a comprehensive overview of the security archi

For detailed information about authentication and authorization mecha

Security Architecture Overview

The TradeX REST Proxy implements a multi-layered security model design

! [Chart 1] (rest-proxy-images/rest-proxy-section14-1.svg)

Sources: [] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/ke

[keys/aaa/tradex/jwt-private.key1-39] (https://github.com/nhsv-tradex/

keys/aaa/tradex/jwt-public.key1-2] (https://github.com/nhsv-tradex/res

Authentication Mechanisms

The system implements dual authentication mechanisms to support diffe

JWT Token Authentication

Client API requests use JWT (JSON Web Token) authentication with RSA

API Key Authentication

Administrative operations utilize API key authentication, providing a

! [Chart 2] (rest-proxy-images/rest-proxy-section14-2.svg)

Sources: [] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/ke

[keys/aaa/tradex/jwt-private.key1-39] (https://github.com/nhsv-tradex/

keys/aaa/tradex/jwt-public.key1-2] (https://github.com/nhsv-tradex/res

Authorization System

The authorization system operates on a scope-based model where permis

Component	Purpose	Data Source
Scope Resolution	Maps URI patterns to permission requirements	
Permission Verification	Validates user permissions against scope	
Dynamic Authorization	Adapts permissions based on context and us	

Security Workflow

The complete security workflow processes each request through multiple

! [Chart 3] (rest-proxy-images/rest-proxy-section14-3.svg)

Sources: [] (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/ke

[keys/aaa/tradex/jwt-private.key1-39] (https://github.com/nhsv-tradex/

keys/aaa/tradex/jwt-public.key1-2] (https://github.com/nhsv-tradex/res

Cryptographic Key Management

The system utilizes RSA key pairs for JWT token signing and verificat

Key Storage Structure

The keys are organized in the following directory structure:

keys/aaa/tradex/ |—— jwt-private.key # RSA private key for JWT signing |—— jwt-
public.key # RSA public key for JWT verification |—— rsa-private.key # Additional RSA
private key |—— rsa-public.key # Additional RSA public key

Key Specifications

The JWT keys implement OpenSSH key format with RSA-2048 encryption:

Key Component	Specification	Usage
Private Key	OpenSSH RSA-2048	JWT token signing operations
Public Key	SSH-RSA format	JWT token signature verification
Key Length	2048 bits	Cryptographic strength
Encoding	Base64 (PEM format)	Standard key representation

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/keys/aaa/tradex/jwt-private.key1-39>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/keys/aaa/tradex/jwt-public.key1-2>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/keys/aaa/tradex/jwt-public.key1-2>)

Security Middleware Integration

The security components are integrated into the Express.js middleware

! [Chart 4] (<rest-proxy-images/rest-proxy-section14-4.svg>)

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/keys/aaa/tradex/jwt-private.key1-39>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/keys/aaa/tradex/jwt-public.key1-2>) (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/keys/aaa/tradex/jwt-public.key1-2>)

The security architecture ensures that the TradeX REST Proxy maintain

Authentication and Authorization

Relevant source files

- [[src/app/middlewares/check-apikey.ts](#)] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts>)
- [[src/app/models/admin/Authentication.ts](#)] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/models/admin/Authentication.ts>)

This document covers the authentication and authorization mechanisms.

For detailed information about the specific middleware implementation,

Authentication Mechanisms Overview

The TradeX REST Proxy implements a dual authentication strategy to support

Authentication Type	Target Users	Validation Method	Authorization
API Key	Admin Applications	Static API Key Comparison	Full Access
JWT Token	Client Applications	RSA Signature Verification	Scoped Access

The system determines which authentication mechanism to apply based on

Admin API Key Authentication

Admin operations use a simple API key authentication mechanism implemented

API Key Validation Process

![[Chart 1](rest-proxy-images/rest-proxy-section15-1.svg)]

The implementation extracts the API key from request headers and performs

[[<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts>]]

[[<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts>]]

Sources: [[<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts>]]

[[<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/middlewares/check-apikey.ts>]]

Admin Authentication Models

The admin authentication system returns structured response objects containing

[[<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/models/admin/Authentication.ts>]]

[[<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74e/src/app/models/admin/Authentication.ts>]]

The `AdminLoginResponse` schema includes:

- `accessToken` - Primary authentication token for API access
- `refreshToken` - Token used to renew expired access tokens
- `userInfo` - Administrative user profile information

Sources: (https://github.com/nhsv-tradex/rest-proxy/blob/9502c74

[src/app/models/admin/Authentication.ts] (https://github.com/nhsv-trad

JWT-Based Client Authentication

Client applications authenticate using JSON Web Tokens (JWT) signed w

JWT Token Structure

![Chart 2](rest-proxy-images/rest-proxy-section15-2.svg)

Token Validation Flow

![Chart 3](rest-proxy-images/rest-proxy-section15-3.svg)

Sources: Based on system architecture and cryptographic key usage

Scope-Based Authorization

Authorization in the TradeX REST Proxy uses a scope-based model where

1. **User Group Membership** - Users belong to specific groups that
2. **Scope Definitions** - API endpoints are mapped to required scop
3. **Dynamic Resolution** - Scopes are resolved at runtime based on

Authorization Decision Matrix

![Chart 4](rest-proxy-images/rest-proxy-section15-4.svg)

Scope Configuration Structure

The scope authorization system relies on configuration data that maps

Configuration Element	Purpose	Location
---	---	---

```
| Scope Definitions | Maps endpoints to required scopes | `scopeData
| OpenAPI Schema | Defines endpoint parameters and validation | `ope
| User Group Mappings | Associates users with permission groups | JW
```

Sources: Based on scope resolution architecture and configuration

Authentication Flow Integration

The complete authentication and authorization flow integrates both me

! [Chart 5] (rest-proxy-images/rest-proxy-section15-5.svg)

This integrated approach ensures that:

- Administrative operations require valid API keys for full system
- Client operations require valid JWT tokens with appropriate scope
- All requests are properly authenticated before being forwarded to
- Authorization failures are handled consistently with appropriate

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

[src/app/middlewares/check-apikey.ts] (<https://github.com/nhsv-tradex/>

src/app/models/admin/Authentication.ts] (<https://github.com/nhsv-trade>

Cryptographic Keys

Relevant source files

- [
 - keys/aaa/tradex/jwt-private.key] (<https://github.com/nhsv-tradex/r>
- [
 - keys/aaa/tradex/jwt-public.key] (<https://github.com/nhsv-tradex/re>
- [
 - keys/aaa/tradex/rsa-private.key] (<https://github.com/nhsv-tradex/r>
- [

keys/aaa/tradex/rsa-public.key] (<https://github.com/nhsv-tradex/re>

This document details the cryptographic key pairs used by the TradeX

For information about how these keys are generated during setup, see

Key Pair Overview

The TradeX REST Proxy system uses two distinct cryptographic key pair

! [Chart 1] (rest-proxy-images/rest-proxy-section16-1.svg)

Sources: [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

[keys/aaa/tradex/jwt-private.key1-39] (<https://github.com/nhsv-tradex/>

keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

keys/aaa/tradex/rsa-private.key1-39] (<https://github.com/nhsv-tradex/r>

keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

JWT Key Pair

The JWT key pair handles JSON Web Token operations for client authent

Component	File Path	Purpose
Private Key	`keys/aaa/tradex/rsa-private.key1-39`	Signs JWT tokens
Public Key	`keys/aaa/tradex/rsa-public.key1-2`	Verifies JWT token

JWT Private Key

The JWT private key is stored in OpenSSH format and used for signing

- **Format:** OpenSSH private key (ssh-rsa)
- **Encoding:** Base64 encoded with OpenSSH headers
- **Usage:** Token signing operations in authentication middleware

JWT Public Key

The corresponding public key enables token verification:

- **Format**: SSH public key format
- **Key Type**: `ssh-rsa`
- **Length**: 3072 bits (based on key data structure)
- **Usage**: Token signature validation

Sources [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
[keys/aaa/tradex/rsa-private.key1-39] (<https://github.com/nhsv-tradex/>
keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

RSA Key Pair

The RSA key pair provides general-purpose encryption and decryption c

Component	File Path	Purpose
Private Key	`keys/aaa/tradex/rsa-private.key`	Decrypts data an
Public Key	`keys/aaa/tradex/rsa-public.key`	Encrypts data and

RSA Private Key

The RSA private key handles sensitive cryptographic operations:

- **Format**: OpenSSH private key format
- **Key Type**: `ssh-rsa`
- **Structure**: Multi-line base64 encoded with OpenSSH delimiters
- **Usage**: Administrative operations requiring encryption/decrypt

RSA Public Key

The RSA public key enables secure communication with external systems

- **Format**: SSH public key format
- **Key Type**: `ssh-rsa`
- **Length**: 3072 bits
- **Usage**: Data encryption and signature verification

Sources [] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>
[keys/aaa/tradex/rsa-private.key1-39] (<https://github.com/nhsv-tradex/>

keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

Key Storage Structure

The cryptographic keys follow a hierarchical directory structure that

! [Chart 2] (rest-proxy-images/rest-proxy-section16-2.svg)

Directory Structure:

- `keys/` - Root directory for all cryptographic keys
- `keys/aaa/` - Authentication, Authorization, Accounting service k
- `keys/aaa/tradex/` - TradeX-specific key storage

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

[keys/aaa/tradex/jwt-private.key1-39] (<https://github.com/nhsv-tradex/>

keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

keys/aaa/tradex/rsa-private.key1-39] (<https://github.com/nhsv-tradex/r>

keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

Key Usage in Authentication Flow

The cryptographic keys integrate into the authentication and authoriz

! [Chart 3] (rest-proxy-images/rest-proxy-section16-3.svg)

Sources:[] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>

[keys/aaa/tradex/jwt-private.key1-39] (<https://github.com/nhsv-tradex/>

keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

keys/aaa/tradex/rsa-private.key1-39] (<https://github.com/nhsv-tradex/r>

keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/res>

Security Considerations

Key Format and Structure

Both key pairs use the OpenSSH key format, which provides:

- **Standardized Format**: Compatible with OpenSSH tools and libraries
- **Base64 Encoding**: Safe transport and storage encoding
- **Header Identification**: Clear key type identification (`ssh-rsa`)
- **Integrity Markers**: Begin/end delimiters for validation

Access Control

The key files require careful access control:

- **Private Keys**: Should have restricted file permissions (600 or less)
- **Public Keys**: Can have broader read permissions (644)
- **Directory Access**: Parent directories should limit write access
- **Process Isolation**: Only authentication services should access keys

Key Rotation Strategy

For production deployments, consider:

- **Regular Rotation**: Establish key rotation schedules
- **Backup Procedures**: Secure backup of key pairs
- **Version Management**: Track key versions during rotation
- **Graceful Transition**: Maintain service availability during key rotation

Sources: [1] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[keys/aaa/tradex/jwt-private.key1-39] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[keys/aaa/tradex/jwt-public.key1-2] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[keys/aaa/tradex/rsa-private.key1-39] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)

[keys/aaa/tradex/rsa-public.key1-2] (<https://github.com/nhsv-tradex/rest-proxy/blob/9502c74>)
